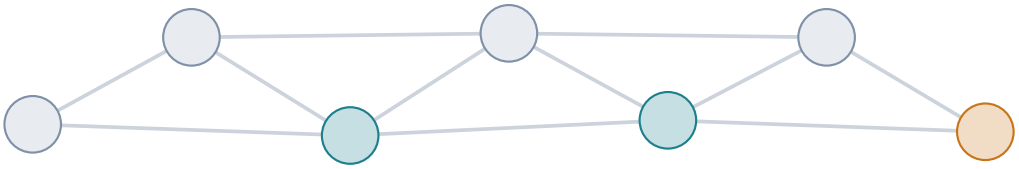
COMPUTER NETWORKS LABORATORY

Experiments 1–10

LAB MSE STUDY GUIDE

EVERY PROGRAM, LINE BY LINE

NS-2 (Tcl) • AWK • Python • TCP and UDP sockets • C



What this guide is

The companion of the lab manual. For *every* program of Experiments 1–10 it shows the code in the same line numbers as the manual and explains what each line does and why, then walks through a dry run, lists the changes an examiner may ask for, and ends with self-check questions.

How to study

1. read the "what and why" and the method; 2. read the cards, covering the explanation and guessing it first; 3. redo the dry run by hand; 4. practise the "change it" table; 5. answer the questions.

Python and AWK outputs were produced by running the programs; NS-2 results come from your manual; facts checked against the sources at the end

Contents

Foundations: Tcl, NS-2, AWK, Python, sockets 1

Experiment 1: Point-to-Point Network, Queue Size and Packet Drops 5

exp1.tcl — line by line 5

exp1.awk — line by line 8

Experiment 2: TCP and UDP Traffic, Packet Counts and Throughput 11

exp2.tcl — line by line 11

exp2.awk — line by line 14

exp2_throughput.awk — line by line 14

Experiment 3: Ethernet LAN, Throughput versus Data Rate and Error Rate 18

exp3.tcl — line by line 18

exp3_error.tcl — line by line 20

exp3.awk — line by line 20

Experiment 4: Congestion Window of TCP Vegas and TCP Reno 23

exp4.tcl — line by line 23

exp4.awk — line by line 26

Experiment 5: Bit Stuffing in the Data-Link Layer 28

exp5.py — line by line 28

Experiment 6: Error Detection Using CRC (CRC-CCITT) 32

exp6.py — line by line 32

Experiment 7: Frame Sorting Technique Used in Buffers 36

exp7.py — line by line 36

Experiment 8: Client-Server File Transfer Using TCP Sockets 39

tcp_server.py — line by line 39

tcp_client.py — line by line 41

Experiment 9: Client-Server Calculator Using UDP Sockets 45

udp_server.py — line by line 45

udp_client.py — line by line 47

Experiment 10: Distance Vector Routing Algorithm 51

exp10.c — line by line 51

Exam Toolkit: Formulas, Commands, Cross-Checks 55

Sources Used to Check the Facts 59

PART A

Foundations: Tcl, NS-2, AWK, Python, sockets

Every program in the lab uses a few ideas again and again. Learn them once on these pages and the line-by-line pages become short and easy. Nothing here is "extra": each rule is used by at least one program of Experiments 1 to 9.

Tcl in ten minutes (Experiments 1 to 4)

NS-2 scripts are written in Tcl (OTcl, its object-oriented extension). Five rules explain almost every line.

1. **Everything is a command.** The first word is the command, the other words are its arguments, separated by spaces. A command ends at the end of the line (or at `;`). `set x 5` is the command `set` with the arguments `x` and `5`.
2. **`$name` is the value of a variable; `set name value` creates or changes it.** After `set ns [new Simulator]`, the text `$ns` means "the thing stored in ns".
3. **`[ ... ]` runs a command and substitutes its result.** In `set n0 [$ns node]` Tcl first runs `$ns node` (it creates a node and returns the node's name) and then performs `set n0 <that name>`.
4. **Quotes and braces group words.** `"..."` groups words and still substitutes `$var` and `[cmd]`; `{...}` groups words and substitutes *nothing*. So `"$cbr0 start"` turns into a string such as `_o12 start` at the moment the line is read, whereas the body `{ ... }` of a `proc` stays untouched until the procedure is called.
5. **`$obj method arguments` calls a method of an object.** `$ns duplex-link ...` asks the simulator object to build a link. A class name such as `Agent/UDP` contains a slash; `new Agent/UDP` creates an object of that class.

Things that appear in the scripts

- `#` starts a comment (it must be where a command could start, i.e. normally at the beginning of a line).
- `proc name {arguments} { body }` defines a procedure. Inside a procedure all variables are local; `global a b` makes the global variables `a` and `b` visible.
- `exec program args &` starts another program (here `nam` and `awk`); the `&` lets `ns` carry on without waiting. `exit 0` ends the script.
- `open file w` opens a file for writing and returns a handle that is stored in a variable; `close $handle` closes it.
- **Two kinds of `set` on objects.** `$tcp set class_ 1` changes one object. `Agent/TCP set packetSize_ 1000` changes the default that is given to the objects of that class created *afterwards*.
- **Units in NS-2.** `1Mb` means 1 Mbit/s, `512Kb` means 512 kbit/s, `10ms` means 10 milliseconds, and packet sizes are in bytes. All simulation times are in seconds.

> **Note.** Tcl is fussy about line layout. A command must stay on one line (the `make-lan` command is the classic victim, see Appendix B of the manual), and a stray word on a line of its own is executed as a command.

How NS-2 runs a script

A script has two phases. In the **build phase** the lines are executed one after another and only *create* objects (nodes, links, agents, applications) and *schedule* actions with `$ns at time "command"`. Nothing is simulated yet. The command `$ns run` starts the **event scheduler**, which takes the scheduled events in time order and executes them; simulated time jumps from event to event. When `finish` is called, the script closes its files and ends.

The journey of one packet (Experiment 1)

A source application hands data to an agent; the agent builds a packet and gives it to the node; the node puts it into the **queue** of the outgoing link; the link transmits it and the next node receives it. The time a link needs is the **transmission time** (size×8 divided by the bandwidth) plus the **propagation delay**. For a 500-byte packet on a 1 Mb/s link with 10 ms delay: 500×8/1 000 000 = 4 ms plus 10 ms = 14 ms.

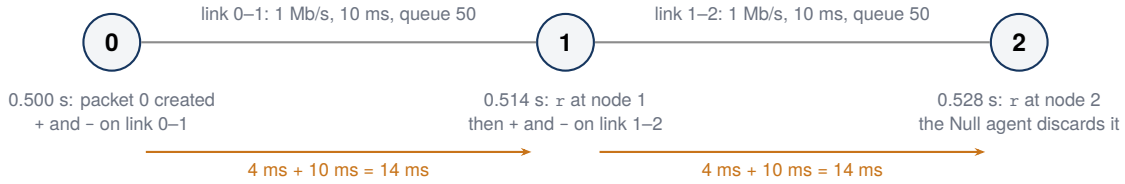


Figure A1. The first packet of Experiment 1 needs 14 ms per link: it reaches node 1 at 0.514 s and node 2 at 0.528 s.

The four events of the trace file

Whenever something happens to a packet on a link, NS-2 writes one line to the trace file. The event letter is the first field: `+` the packet is **enqueued** (put into the queue), `-` it is **dequeued** (transmission starts), `r` it is **received** at the far end of the link, `d` it is **dropped** (the queue was full). On a LAN the letter `h` marks a hop. Every line has twelve fields, which AWK calls `$1` to `$12`.

Reading one trace line

```
r 0.514 0 1 cbr 500 ------- 0 0.0 2.0 0 0
$1 $2    $3 $4 $5 $6  $7    $8 $9 $10 $11 $12
```

Field	Value here	Meaning
`$1`	`r`	event: received at the far end of a link
`$2`	`0.514`	time of the event in seconds (0.5 + 4 ms + 10 ms)
`$3`, `$4`	`0`, `1`	the packet travelled on the link from node 0 to node 1
`$5`	`cbr`	packet type (`cbr` = traffic of the CBR application; also `tcp`, `ack`)
`$6`	`500`	size in bytes (in recent NS-2 versions a TCP data packet of 1000 bytes shows as 1040: the 40-byte TCP/IP header is added)
`$7`	`-------`	flags (not used in these experiments)
`$8`	`0`	flow id (`class_`); sets the colour in NAM
`$9`, `$10`	`0.0`, `2.0`	source and destination address written as *node.port*: from node 0 to node 2
`$11`	`0`	sequence number of the packet within its flow
`$12`	`0`	unique packet id

AWK in five minutes (Experiments 1 to 4)

AWK reads a text file line by line and runs your program on every line. The program is a list of *pattern* { *action* } pairs; a block without a pattern runs for every line.

- Each input line is split into **fields** at blanks: `$1` is the first, `$2` the second, ..., `$NF` the last; `$0` is the whole line.
- `BEGIN { ... }` runs once **before** the first line; `END { ... }` runs once **after** the last line. They are used to set up and to print totals.
- Variables need no declaration. A variable that was never assigned is `0` as a number and empty as a string, so `count++` works at once.
- `$1 == "d"` compares with a string constant; `time > stopTime` compares numbers (fields that look like numbers are compared as numbers).
- `printf("format", values)` prints formatted text: `%d` integer, `%f` real number, `%.2f` two decimals, `%s` string, `\t` tab, `\n` new line. `print` is the short form.
- You run a program with `awk -f exp1.awk out.tr`: the program file after `-f`, the data file last. `> file` sends the output to a file.

> **Careful.** In AWK the `{` of a `BEGIN` block must be on the same line as the word `BEGIN`, and strings use double quotes only (Appendix B of the manual).

Python used in Experiments 5 to 9

- `input("text")` shows the text and returns what the user typed as a **string**; `int(...)` and `float(...)` convert it to a number.
- Lists: `data = [1, 0, 1]`, `data.append(0)`, `data.copy()` (an independent copy), `len(data)`, `data[i]` (counting from 0), `data[-1]` (the last item).
- Slicing: `msg[i:i+3]` is the part of `msg` from index `i` up to (not including) `i+3`.
- `for i in range(n)` repeats with `i = 0, 1, ..., n-1`; `for bit in text` visits the characters of a string one by one.
- `a ^ b` is the bitwise **XOR** (1 when the bits differ); `a ^= b` means `a = a ^ b`.
- A list comprehension `[expr for i in range(n)]` builds a list in one line; an f-string `f"Enter bit {i + 1}: "` puts the value of an expression into the text.
- `enumerate(list, start=1)` pairs each item with a counter; tuples such as `(2, 'MIT')` are compared item by item.
- `bytes` and `str`: a socket sends and receives **bytes**. `text.encode()` turns a string into bytes (UTF-8), `data.decode()` turns bytes back into a string. A literal `b"abc"` is a bytes value.
- `try: ... except ValueError: ...` runs the first block and, if that error occurs, the second one instead of stopping the program. `with open(name) as f:` closes the file automatically.

Sockets in one page

A **socket** is one end of a communication channel between two programs. Programs find each other through an **IP address** (which computer) and a **port number** (which program on that computer, 0–65535). `127.0.0.1` ("localhost") always means "this computer", which is why one computer is enough for the lab.

	TCP (Experiment 8)	**UDP (Experiment 9)**
socket type	`SOCK_STREAM`	`SOCK_DGRAM`
connection	set up first (handshake)	none
data unit	a byte *stream*: no message boundaries	separate *datagrams*: boundaries kept
reliability	lost data is re-sent, order kept	datagrams may be lost or reordered
server calls	`bind, listen, accept, recv, sendall`	`bind, recvfrom, sendto`
client calls	`connect, sendall, recv`	`sendto, recvfrom`

EXPERIMENT 1

Experiment 1: Point-to-Point Network, Queue Size and Packet Drops

What the program does and why

Three nodes are joined in a chain, `n0 - n1 - n2`. A UDP source on `n0` sends 500-byte packets every 5 ms (800 kb/s) to a sink on `n2` for 4 seconds. If a link is **slower than 800 kb/s**, packets arrive faster than they can be sent, the **queue** in front of the link fills up, and a full DropTail queue discards every further packet. The experiment varies the bandwidth and the queue limit and **counts the dropped packets** in the trace file with AWK.

The method in six steps

1. Create the simulator and the two trace files (lines 2–10).
2. Write the `finish` procedure that closes the files and starts NAM (lines 12–20).
3. Create the nodes and the links with bandwidth, delay and queue (lines 22–37).
4. Create the UDP agent on `n0`, the Null agent on `n2`, and connect them (lines 39–47).
5. Create the CBR application and attach it to the UDP agent (lines 49–53).
6. Schedule start, stop and finish, then run (lines 55–61). Afterwards `exp1.awk` counts the lines that begin with `d`.

Lines	Block of the program
1–2	header comment, the simulator object
4–10	the two trace files (NAM and text)
12–20	the `finish` procedure
22–25	the three nodes
27–37	links (bandwidth, delay, queue type), NAM layout, queue limits
39–47	UDP sender, Null receiver, connection
49–53	CBR traffic source
55–61	start/stop/finish events and `$ns run`

Line by line: exp1.tcl

```
1 # gedit exp1.tcl
```

A **comment**: everything after `#` is ignored. `gedit exp1.tcl` only records which command was used to open the file in the editor. Writing the file name in the first line helps you recognise the program.

```
2 set ns [new Simulator]
```

Creates the **simulator object** and stores it in the variable `ns`. This object owns the event scheduler (the clock, starting at 0) and offers all the commands used below: `$ns node`, `$ns duplex-link`, `$ns at`, `$ns run` ... Every NS-2 script begins with this line. `new Simulator` is the command; the square brackets make Tcl run it first and pass its result (the new object) to `set`.

```
4  # open a new file for NAMTRACE
5  set nf [open out.nam w]
6  $ns namtrace-all $nf
```

Prepare the file for the **animation**. Line 5 opens (creates) `out.nam` for writing (`w`); `open` returns a *file handle*, kept in `nf`. Line 6 tells the simulator to write every event that NAM needs into that file. Without line 6 the file stays empty and NAM shows nothing.

```
8  # open a new file to log TRACE
9  set tf [open out.tr w]
10 $ns trace-all $tf
```

The same for the **text trace** that AWK will read: `tf` is the handle of `out.tr`, and `trace-all $tf` makes the simulator write one line (`+`, `-`, `r` or `d`) for every packet event on every link. All the measurements of the experiment come from this file.

```
12 # body of finish procedure
13 proc finish {} {
```

Line 12 is a comment. Line 13 **defines a procedure** called `finish`: `proc name arguments body`. The argument list `{}` is empty and the body is the block between the braces (lines 14–19). Defining a procedure does *not* run it; it runs when the scheduler reaches the event of line 59.

```
14     global ns nf tf
```

Inside a procedure, variables are *local*. `ns`, `nf` and `tf` were created outside, so they must be declared `global` before they can be used. Forgetting this gives the error `can't read "tf": no such variable` (Appendix B of the manual).

```
15     $ns flush-trace
```

The simulator collects trace data in memory and writes it in blocks. `flush-trace` forces everything that is still waiting to be written to the files, so that nothing is lost.

```
16     close $nf
17     close $tf
```

Close the two files. Closing completes them; AWK and NAM need finished files.

```
18     exec nam out.nam &
```

`exec` starts an operating-system program from inside Tcl: here the **Network Animator** on `out.nam`. The `&` at the end starts it in the background, so `ns` does not wait until you close the NAM window.

```
19     exit 0
20 }
```

`exit 0` ends the `ns` program (0 = no error). Line 20 closes the body of the procedure.

```
22 # Create nodes
23 set n0 [$ns node]
24 set n1 [$ns node]
25 set n2 [$ns node]
```

Three **nodes**. `$ns node` creates a node and returns its name; `set n0 [...]` stores it. The nodes are numbered 0, 1, 2 in the order of creation, and these numbers appear in the trace file (`$3` and `$4`). The names `n0`, `n1`, `n2` are only for your convenience.

```
27 # Create links between nodes
28 $ns duplex-link $n0 $n1 1Mb 10ms DropTail
29 $ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

Line 28 reads: "join `n0` and `n1` by a duplex link of 1 Mb/s and 10 ms delay with a DropTail queue." **Duplex** means both directions: NS-2 creates two simplex links, n0→n1 and n1→n0, each with its own queue. `1Mb` is the **bandwidth** (the value you change in the experiment: 0.25Mb, 0.5Mb, ...), `10ms` the **propagation delay**, and `DropTail` the **queue discipline**: a first-in-first-out buffer that discards the *arriving* packet when it is full. Line 29 does the same for n1 and n2, so the network is the chain 0 – 1 –

2.

```
31 # set orientation
32 $ns duplex-link-op $n0 $n1 orient right-up
33 $ns duplex-link-op $n1 $n2 orient right-down
```

Only for **NAM**: they tell it where to draw the nodes (`n1` to the upper right of `n0`, `n2` to the lower right of `n1`). They have no effect on the simulation or on the trace file.

```
35 # set queue limit-default is 50 packets
36 $ns queue-limit $n0 $n1 50
37 $ns queue-limit $n1 $n2 50
```

`queue-limit n1 n2 limit` sets the **buffer size** of the link *from* the first node *to* the second, counted in packets. The comment says 50 is the default, so these lines change nothing as they stand; they are the lines to edit when the task says *set the queue size*. The setting is **directional**: line 36 is the link n0→n1, line 37 the link n1→n2. With DropTail a limit of n really holds at most $n - 1$ waiting packets (NS-2 manual, section 7.2).

```
39 # Create transport agent
40 # Sender
41 set udp0 [new Agent/UDP]
42 $ns attach-agent $n0 $udp0
```

A **UDP agent** is the sending end of a transport protocol. Line 41 creates it; line 42 **attaches** it to node `n0` (an agent that is not attached to a node cannot send). UDP only wraps the data into packets: no handshake, no acknowledgements, no retransmission.

```
44 # Receiver
45 set null0 [new Agent/Null]
46 $ns attach-agent $n2 $null0
47 $ns connect $udp0 $null0
```

The receiving end is a **Null agent**: it accepts packets and throws them away, which is all a UDP sink has to do. Line 46 attaches it to `n2`. Line 47, `connect $udp0 $null0`, tells the sender *where its packets must go* (it sets the destination of `udp0` to the Null agent). Without `connect` the sender would not know its destination.

```
49 # Create Application to generate traffic
50 set cbr0 [new Application/Traffic/CBR]
51 $cbr0 set packetSize_ 500
52 $cbr0 set interval_ 0.005
53 $cbr0 attach-agent $udp0
```

An **application** generates the data; `Application/Traffic/CBR` makes a *constant bit rate* stream. `packetSize_` is the size of each packet in bytes (500), `interval_` the time between two packets in seconds (0.005). The offered rate is therefore $500 \times 8 \,/\, 0.005 = 800\,000$ bit/s = **800 kb/s**: less than 1 Mb/s (no loss) but more than 0.75 Mb/s (loss). Line 53 attaches the application to the UDP agent. NS-2 lets you give either `rate_` or `interval_`.

Careful. A misspelt option such as `packet Size_` is silently accepted and the default of 210 bytes is used (Appendix B of the manual).

```
55 # Start and stop generating traffic
56 $ns at 0.5 "$cbr0start"
57 $ns at 4.5 "$cbr0stop"
```

`$ns at time "command"` **schedules** an action: at 0.5 s the command `$cbr0 start` is executed, at 4.5 s `$cbr0 stop`. The double quotes make Tcl substitute `$cbr0` *now*, so the scheduler stores a string such as `_o17 start`. The source thus sends for 4 seconds, one packet every 5 ms: **801 packets** in the trace (the manual's UDP count of 4806 = 801 × 6 lines shows that the packet at 4.5 s is sent too).

```
59 $ns at 5.0 "finish"
```

Schedules the `finish` procedure at 5.0 s, half a second after the source has stopped, so that the last packets still travelling on the links can arrive before the files are closed.

```
61 $ns run
```

`$ns run` **starts the simulation**: the scheduler executes the events in time order (0.5 s start, the packet events, 4.5 s stop, 5.0 s finish) until `finish` calls `exit`. All earlier lines only *prepared* the experiment.

Line by line: exp1.awk

```
1 # gedit exp1.awk
```

A comment (in AWK also `#`).

```
2 BEGIN { count = 0; }
```

`BEGIN { ... }` runs **once, before the first line of the trace is read**. It sets the counter `count` to 0. (AWK would start it at 0 anyway; writing it makes the program clearer.)

```
3 {
4     if( $1 == "d")
```

Line 3 opens the **main block**: it has no pattern in front of it, so it is executed for **every line** of `out.tr`. Line 4 tests the first field: `$1` is the event letter, and only lines of *dropped* packets (`d`) pass the test.

```
5     {
6         count++;
7         printf("%s\t %s\n", $5 , $11 );
8     }
9 }
```

For each dropped packet: `count++` adds 1; `printf("%s\t %s\n", $5, $11)` prints the packet type (`cbr`) and the **sequence number** (`$11`) of the lost packet, so you can see *which* packets were lost (for 0.5 Mb the first lost one is `cbr 131`). Lines 8 and 9 close the `if` and the main block.

```
10 END { printf("The number of packets dropped =%d \n", count ); }
```

`END { ... }` runs **once, after the last line**: it prints the total. `%d` prints `count` as an integer.

Dry run: what the program does with a trace

Here is a tiny trace (made by hand in the real NS-2 format; two packets are dropped) and the real output of `awk -f exp1.awk` on it.

mini1.tr (11 lines) and the output of awk -f exp1.awk mini1.tr

```
+ 0.5 0 1 cbr 500 ------- 0 0.0 2.0 0 0
- 0.5 0 1 cbr 500 ------- 0 0.0 2.0 0 0
+ 0.505 0 1 cbr 500 ------- 0 0.0 2.0 1 1
+ 0.51 0 1 cbr 500 ------- 0 0.0 2.0 2 2
d 0.51 0 1 cbr 500 ------- 0 0.0 2.0 2 2      <-- $1 == "d": count = 1, prints  cbr 2
r 0.514 0 1 cbr 500 ------- 0 0.0 2.0 0 0
+ 0.514 1 2 cbr 500 ------- 0 0.0 2.0 0 0
- 0.514 1 2 cbr 500 ------- 0 0.0 2.0 0 0
+ 0.515 0 1 cbr 500 ------- 0 0.0 2.0 3 3
d 0.515 0 1 cbr 500 ------- 0 0.0 2.0 3 3      <-- $1 == "d": count = 2, prints  cbr 3
r 0.528 1 2 cbr 500 ------- 0 0.0 2.0 0 0

$ awk -f exp1.awk mini1.tr
cbr^^I 2
```

```
cbr^^I 3
The number of packets dropped =2
```

Why packets are dropped: the arithmetic

Every packet needs a **service time** of size×8/bandwidth on the link: 4 ms at 1 Mb/s, 5.33 ms at 0.75 Mb/s, 8 ms at 0.5 Mb/s, 16 ms at 0.25 Mb/s. A new packet arrives every 5 ms. At 1 Mb/s the link is faster than the source (4 ms < 5 ms), so the queue stays empty. Slower links fall behind by the difference; the queue grows until it holds *limit − 1* packets, and from then on every packet that arrives while it is full is lost. In 4 seconds the link can send about (C/L)·T packets (C bandwidth, L = 4000 bit packet size, T = 4 s), so

drops $\approx N - \frac{C}{L}T -$ limit, with N = 801 packets sent.

Setting	NS-2 (your manual)	formula	20-line model
0.25 Mb, limit 50	502	501	502
0.5 Mb, limit 50	252	251	252
0.75 Mb, limit 50	2	1	1
1 Mb, limit 50	0	0	0
0.75 Mb, limit 40	12	11	11
0.75 Mb, limit 30	22	21	21
0.75 Mb, limit 20	32	31	31
0.75 Mb, limit 10	42	41	41

Figure A2. Independent check of the manual's Experiment 1 results: the formula and a small queue model written in Python (it follows the NS-2 rule that a DropTail queue holds at most limit minus one waiting packets) agree with the NS-2 numbers to within one packet.

This also explains the shape of the two graphs in the manual: the drops fall **linearly** when the bandwidth or the limit increases, and they vanish as soon as the link is faster than the source.

Change it: what the examiner may ask

Task	Change	Where
other bandwidth	`1Mb` → `0.5Mb` on both links	lines 28, 29
other propagation delay	`10ms` → `50ms`	lines 28, 29
other queue size	`50` → `10`	lines 36, 37
other packet size or rate	`packetSize_`, `interval_` (or `rate_ 1Mb`)	lines 51, 52
send longer or shorter	the times of `start`, `stop`, `finish`	lines 56, 57, 59
other queue type	`DropTail` → `RED` or `SFQ`	lines 28, 29
drops on one link only	`if($1=="d" && $3==0)`	exp1.awk line 4
list the lost packets	already printed by `printf` (type and sequence)	exp1.awk line 7

In the exam. After changing a value, run `ns exp1.tcl` again and then `awk -f exp1.awk out.tr`. Save each result (for example in `plot_graph_1` as two columns *bandwidth drops*) and plot it with `xgraph`. If the drop count is 0, the link is faster than the source; you may have set the bandwidth in only one of the two `duplex-link` lines (the *slowest* link decides).

Check yourself

1. Which line, if deleted, leaves NAM with nothing to show? (*line 6, `namtrace-all`*)
2. What does `$ns connect $udp0 $null0` do, and what happens without it? (*it sets the destination of the sender to the sink; without it the sender does not know where to send*)
3. Why is the offered rate exactly 800 kb/s? (*500 B × 8 = 4000 bit every 0.005 s*)
4. Why do lines 36 and 37 change nothing in the program as printed? (*50 is already the default limit*)
5. The bandwidth is 0.75 Mb on the first link and 1 Mb on the second. Where are packets dropped? (*only in front of the first link, the slower one; the second link receives at most 0.75 Mb/s*)
6. Why does `count` need no declaration in AWK? (*an unused variable is 0*)

EXPERIMENT 2

Experiment 2: TCP and UDP Traffic, Packet Counts and Throughput

What the program does and why

Four nodes: `n0` and `n1` are connected to the router `n2`, and `n2` is connected to the receiver `n3`. A **TCP** flow (FTP) runs from `n0` to `n3` and a **UDP** flow (CBR, 800 kb/s) from `n1` to `n3`. Both flows must pass through the link `n2-n3`, the **bottleneck**. UDP never slows down, so TCP only gets what is left over; when the bandwidth of the links is increased, TCP gains much more than UDP. Two AWK programs measure this: `exp2.awk` counts trace lines of each protocol, `exp2_throughput.awk` computes the throughput over time.

The method

1. Build the simulator, trace files and `finish` as in Experiment 1 (lines 1–19).
2. Create four nodes and three duplex links; set queue limits (lines 21–45).
3. Create the TCP sender on `n0`, the TCP sink on `n3`, connect them and attach an FTP source (lines 47–62).
4. Create the UDP sender on `n1`, the Null sink on `n3`, connect them and attach the CBR source (lines 64–78).
5. Schedule the sources and run (lines 80–90). Run the two AWK programs on `out.tr`.

Lines	Block of the program
1–19	simulator, trace files, `finish` (identical to Experiment 1)
21–30	four nodes and their NAM labels
32–45	three links, NAM layout, queue limits
47–62	TCP sender, TCP sink, FTP application, TCP packet size
64–78	UDP sender, Null sink, CBR application
80–90	start/stop events, finish, run

Line by line: exp2.tcl

```
1 set ns [new Simulator]
2
3 # open a new file for NAMTRACE
4 set nf [open out.nam w]
5 $ns namtrace-all $nf
6
7 # open a new file to log TRACE
8 set tf [open out.tr w]
9 $ns trace-all $tf
```

Same as Experiment 1, lines 2–10: the simulator object `ns`, the NAM file (`nf`, `namtrace-all`) and the text trace file (`tf`, `trace-all`).

```
11 # body of finish procedure
12 proc finish {} {
13     global ns nf tf
14     $ns flush-trace
15     close $nf
16     close $tf
17     exec nam out.nam &
18     exit 0
19 }
```

Same as Experiment 1, lines 12–20: the `finish` procedure declares the global variables, flushes and closes the two files, starts NAM in the background and ends the program. It runs at 5.0 s (line 89).

```
21 # Create 4 nodes
22 set n0 [$ns node]
23 set n1 [$ns node]
24 set n2 [$ns node]
25 set n3 [$ns node]
```

Four nodes, numbered 0 to 3 in the order of creation (`$3` and `$4` of the trace lines). `n0` and `n1` are the senders, `n2` the router, `n3` the receiver.

```
27 # set label
28 $n0 label "TCP"
29 $n1 label "UDP"
30 $n2 label "Receiver"
```

`$n0 label "TCP"` writes a **text label** next to the node in NAM. Labels are decoration only and do not influence the result. (`n2` is really the router and `n3` the receiver; the label "Receiver" on `n2` is just a name.)

```
32 # Create links between nodes
33 $ns duplex-link $n0 $n2 1Mb 10ms DropTail
34 $ns duplex-link $n1 $n2 1Mb 10ms DropTail
35 $ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

Three duplex links, all 1 Mb/s, 10 ms, DropTail: `n0`–`n2`, `n1`–`n2`, `n2`–`n3`. The link **`n2`–`n3`** is shared by both flows, so it is the bottleneck. The bandwidth `1Mb` is the value you change in this experiment (1, 1.25, 1.5, 1.75, 2 Mb in the manual).

```
37 # set orientation
38 $ns duplex-link-op $n0 $n2 orient right-down
39 $ns duplex-link-op $n1 $n2 orient right-up
40 $ns duplex-link-op $n2 $n3 orient right
```

NAM layout only (where the nodes are drawn); no effect on the simulation.

```
42 # set the queue limit-default is 50 packets
43 $ns queue-limit $n0 $n2 50
44 $ns queue-limit $n1 $n2 50
45 $ns queue-limit $n2 $n3 50
```

Queue limits of 50 packets, one per *direction* given: `n0`→`n2`, `n1`→`n2` and `n2`→`n3`. The queue of `n2`→`n3` is where TCP and UDP packets meet and compete for space; packets are dropped there when it is full.

```
47 # Create TCP agent b/w node 0 and node 3
48 # Sender
49 set tcp0 [new Agent/TCP]
50 $ns attach-agent $n0 $tcp0
51
52 # Receiver
53 set sink0 [new Agent/TCPSink]
54 $ns attach-agent $n3 $sink0
55 $ns connect $tcp0 $sink0
```

The **TCP sender** (line 49, `Agent/TCP`, the plain Tahoe version) is attached to `n0`. The **TCP sink** (line 53, `Agent/TCPSink`) is attached to `n3`; it receives the data and sends **acknowledgements** back, which UDP's

Null agent never does. Line 55 `connect` joins the two: the sender now knows its destination. TCP is reliable: the sink acknowledges the data and the sender retransmits what is lost.

```
57 # Create FTP Application for TCP Agent
58 set ftp0 [new Application/FTP]
59 $ftp0 attach-agent $tcp0
```

`Application/FTP` simulates a **file transfer**: an unlimited supply of data, which TCP sends as fast as its window and the network allow. Line 59 attaches the FTP source to the TCP agent `tcp0`.

```
61 # Specify TCP packet size
62 Agent/TCP set packetSize_ 1000
```

`Agent/TCP set packetSize_ 1000` sets the **default** payload of 1000 bytes for TCP agents created *after* this line. `tcp0` already exists (line 49), so the line does not change it; here it makes no difference because 1000 bytes is also the default value. In the trace of recent NS-2 versions a data packet shows as 1040 bytes (1000 + the 40 bytes of TCP/IP header that the TCP agent adds); UDP packets have no header added, which is why the CBR packets keep their 500 bytes. To change the size of this agent write `$tcp0 set packetSize_ 500`.

```
64 # Create UDP Agent b/w node1 and node3
65 # Sender
66 set udp0 [new Agent/UDP]
67 $ns attach-agent $n1 $udp0
```

The **UDP sender** on `n1`, as in Experiment 1: create the agent (line 66) and attach it to the node (line 67).

```
69 # Receiver
70 set null0 [new Agent/Null]
71 $ns attach-agent $n3 $null0
72 $ns connect $udp0 $null0
```

The **Null sink** on `n3` and `connect $udp0 $null0`: the sender's packets are addressed to the Null agent on `n3`. Both flows now end at `n3` but on different agents.

```
74 # Create CBR Application for UDP Agent
75 set cbr0 [new Application/Traffic/CBR]
76 $cbr0 set packetSize_ 500
77 $cbr0 set interval_ 0.005
78 $cbr0 attach-agent $udp0
```

The **CBR source**: 500-byte packets every 5 ms = 800 kb/s, attached to the UDP agent (as in Experiment 1, lines 49–53). This stream does not react to losses or to the bandwidth.

```
80 # Start and Stop FTP Traffic
81 $ns at 0.75 "$ftp0start"
82 $ns at 4.75 "$ftp0stop"
83
84 # Start and stop CBR traffic
85 $ns at 0.5 "$cbr0start"
86 $ns at 4.5 "$cbr0stop"
```

Schedule the sources. FTP runs from 0.75 s to 4.75 s, CBR from 0.5 s to 4.5 s. CBR starts first, so for a quarter of a second UDP has the link alone; then TCP starts with its slow start and takes what the CBR flow leaves free.

```
88 # Stop the simulations
89 $ns at 5.0 "finish"
90 $ns run
```

`finish` at 5.0 s (after both sources have stopped and the last packets have arrived) and `$ns run` to start the simulation.

Line by line: exp2.awk

```
# gedit exp2.awk
```

Comment line.

```
BEGIN {
    tcp_count = 0;
    udp_count = 0;
}
```

`BEGIN`: before the first line, set the two counters to 0, one for TCP and one for UDP packets.

```
{
    if( $5 == "cbr")
        udp_count++;
    else if( $5 == "tcp")
        tcp_count++;
}
```

The main block runs for every trace line. `$5` is the **packet type**. If it is `cbr` (the UDP flow) `udp_count` is increased; otherwise, if it is `tcp` (TCP data), `tcp_count` is increased. Lines of other types (`ack`, the acknowledgements of TCP) are not counted. Note that the program counts **trace lines**, not packets: every packet creates several lines (see below).

```
END {
    printf("The number of packets by TCP =%d\n", tcp_count );
    printf("The number of packets by UDP =%d\n", udp_count );
}
```

`END`: after the last line, print the two totals.

How to read 4806. Without drops a UDP packet makes 6 trace lines: `+ - r` on the link `n1-n2` and `+ - r` on the link `n2-n3`. The source sends 801 packets, and $801 \times 6 = \mathbf{4806}$, exactly the count at 1.75 Mb and 2 Mb in the manual. At 1 Mb the count is 4786: 20 lines fewer. A packet that is dropped at the queue of `n2-n3` leaves only `+` and `d` on that link (5 lines instead of 6), so about **20 UDP packets** were lost at 1 Mb and none at 1.75 or 2 Mb. TCP has no fixed value because it sends as much as the network allows.

Line by line: exp2_throughput.awk

```
# gedit exp2_throughput.awk
```

Comment line.

```
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
```

`BEGIN`: `sSize` collects the **bytes received**. `startTime = 5.0` is deliberately *larger* than any time in the simulation, so the first enqueue will make it smaller; `stopTime = 0.1` is deliberately *smaller*, so the first receive will make it larger. `Tput` holds the current throughput.

```
{
    event = $1;
    time = $2;
    size = $6;
```

The main block (every trace line). Three fields are copied into readable variables: `event = $1` (the letter), `time = $2` (the time in seconds) and `size = $6` (the packet size in bytes).

```
    if( event == "+")
    {
        if ( time < startTime ) {
            startTime = time;
        }
    }
```

If the event is `+` (a packet was enqueued) and its time is **smaller than** `startTime`, remember it: after all lines `startTime` is the time of the **first packet sent** (0.5 s).

```
    if( event == "r")
    {
        if( time > stopTime )
        {
            stopTime = time;
        }
        sSize += size;
    }
```

If the event is `r` (a packet was received): if its time is **larger than** `stopTime`, remember it, so `stopTime` ends as the time of the **last receive**; and add the packet size to `sSize` (`+=` adds to the variable). Every receive on every link counts, so a packet that crosses two links is added twice: this is the **aggregate throughput of the whole network**.

```
    Tput = ( sSize / ( stopTime - startTime )) * (8/1000) ;
```

The throughput so far: `sSize / (stopTime - startTime)` is bytes per second; multiplied by `8/1000` it becomes **kilobits per second** (×8 bits per byte, ÷1000).

```
    printf("%f\t %.2f\n", time , Tput );
```

Prints the time of the current line and the throughput so far, **for every line**: the file therefore holds one `time throughput` pair per trace line, and `xgraph` draws the curve of throughput against time.

```
}
END {
}
```

Line 28 closes the main block; `END { }` is empty because everything is printed line by line.

Why does the file begin with `-0.00`? Before the first `r` line, `stopTime` is still 0.1 and `startTime` is already 0.5, so the formula divides 0 by $(0.1 - 0.5) = -0.4$ and gives minus zero. It is harmless (the manual mentions it). Below is the real output of both scripts on a small trace that follows two packets (a UDP packet and a TCP packet and its acknowledgement).

mini2.tr (13 trace lines) and the output of the two AWK programs

```
+ 0.5 1 2 cbr 500 ------- 0 1.0 3.0 0 0
- 0.5 1 2 cbr 500 ------- 0 1.0 3.0 0 0
r 0.512 1 2 cbr 500 ------- 0 1.0 3.0 0 0
+ 0.512 2 3 cbr 500 ------- 0 1.0 3.0 0 0
- 0.512 2 3 cbr 500 ------- 0 1.0 3.0 0 0
r 0.524 2 3 cbr 500 ------- 0 1.0 3.0 0 0
+ 0.75 0 2 tcp 1040 ------- 1 0.0 3.0 0 1
- 0.75 0 2 tcp 1040 ------- 1 0.0 3.0 0 1
r 0.77 0 2 tcp 1040 ------- 1 0.0 3.0 0 1
+ 0.77 2 3 tcp 1040 ------- 1 0.0 3.0 0 1
- 0.77 2 3 tcp 1040 ------- 1 0.0 3.0 0 1
```

```
r 0.79 2 3 tcp 1040 ------- 1 0.0 3.0 0 1
+ 0.79 3 2 ack 40 ------- 1 3.0 0.0 0 2

$ awk -f exp2.awk mini2.tr
The number of packets by TCP =6
The number of packets by UDP =6

$ awk -f exp2_throughput.awk mini2.tr          (time, throughput in kb/s)
0.500000^^I -0.00
0.500000^^I -0.00
0.512000^^I 333.33
0.512000^^I 333.33
0.512000^^I 333.33
0.524000^^I 333.33
0.750000^^I 333.33
0.750000^^I 333.33
0.770000^^I 60.44
0.770000^^I 60.44
0.770000^^I 60.44
0.790000^^I 84.97
0.790000^^I 84.97
```

Check the first values by hand. startTime = 0.5. At the first `r` (0.512 s, 500 bytes): 500 / (0.512 − 0.5) = 41 667 bytes/s, × 8 / 1000 = **333.33 kb/s**. The `r` at 0.524 s adds another 500 bytes: 1000 / 0.024 gives the same 333.33. The TCP packet is received at 0.77 s: (1000 + 1040) / (0.77 − 0.5) × 0.008 = **60.44 kb/s**, and the second TCP receive at 0.79 s gives 3080 / 0.29 × 0.008 = **84.97 kb/s**. On a real trace the curve rises quickly after 0.5 s and flattens at the aggregate throughput, as in Figure 6 of the manual. Both counts are 6: each packet makes six `cbr` or `tcp` lines, and the `ack` line is not counted.

Change it: what the examiner may ask

Task	Change	Where
other link bandwidth	`1Mb` → `2Mb` (all three links)	lines 33–35
make `n2--n3` slower than the rest	change only line 35	line 35
other UDP load	`interval_` or `packetSize_` of the CBR source	lines 76, 77
other TCP packet size	`$tcp0 set packetSize_ 500` (after line 49)	line 62
other TCP window	`$tcp0 set window_ 40`	after line 49
different start/stop times	the four `at` lines	lines 81–86
count `ack` packets too	add `else if($5=="ack") ack_count++`	exp2.awk line 9
throughput of the TCP flow only	test `$5=="tcp"` in the `r` branch	throughput.awk line 18

In the exam. For the table of the manual: change `1Mb` in lines 33–35, run `ns exp2.tcl`, then `awk -f exp2.awk out.tr` and note the two numbers; repeat for each bandwidth and plot `plot_tcp` and `plot_udp` with `xgraph`. For the throughput curve run `awk -f exp2_throughput.awk out.tr > plot_throughput` and `xgraph plot_throughput`.

Check yourself

1. Why is `n2-n3` the bottleneck? (*the traffic of both flows must cross it*)
2. Why does the UDP count stay almost constant while the TCP count grows with the bandwidth? (*CBR sends 800 kb/s whatever happens; TCP adapts its window to the free capacity*)
3. What does `TCPSink` do that `Null` does not? (*it sends acknowledgements*)
4. Why does the throughput script start `startTime` at 5.0 and `stopTime` at 0.1? (*so that the first real values replace them*)
5. Why is a TCP data packet 1040 bytes in the trace although `packetSize_` is 1000? (*the TCP agent adds 40 bytes of TCP/IP header*)
6. What is counted by `tcp_count`: packets or trace lines? (*trace lines; several per packet*)

EXPERIMENT 3

Experiment 3: Ethernet LAN, Throughput versus Data Rate and Error Rate

What the program does and why

Seven nodes. `n0` is the TCP source; a point-to-point link of 3 Mb/s joins it to `n1`, which belongs to an **Ethernet LAN** made of `n1`–`n6`. All LAN nodes share one channel (a *bus*) and use **CSMA/CD**: a node listens before sending and, if two nodes send at the same time, both stop and try again after a random time. An FTP source on `n0` sends to the TCP sink on `n6`. The experiment changes the **LAN data rate** (the `2Mb` in `make-lan`) and later adds an **error model** (packets are lost with probability p) and compares the throughput. The AWK script runs automatically at the end of the simulation and writes `exp3.graph`.

The method

1. Build simulator, trace files and `finish`; this time `finish` also starts `awk` (lines 1–15).
2. Create the seven nodes, label and colour them for NAM (lines 17–35).
3. Create the point-to-point link `n0-n1` and the LAN `n1-n6` with `make-lan` (lines 37–40).
4. Create the TCP sender, the TCP sink, connect them, attach FTP (lines 41–50).
5. Schedule FTP from 1.0 s to 5.0 s, finish at 5.5 s, run (lines 52–55).
6. Plot `exp3.graph` with `xgraph`; repeat with other data rates and with the error model.

Line by line: exp3.tcl

```
1  set ns [new Simulator]
2  set namfile [open exp3.nam w]
3  $ns namtrace-all $namfile
4  set tracefile [open exp3.tr w]
5  $ns trace-all $tracefile
```

The simulator and the two trace files, as in Experiment 1 (lines 2–10) but with the handle names `namfile` and `tracefile`: `exp3.nam` for the animation (`namtrace-all`) and `exp3.tr` for the text trace (`trace-all`).

```
7  proc finish {} {
8      global ns namfile tracefile
```

The `finish` procedure, as before. `global ns namfile tracefile` is needed because the procedure uses these three variables that were created outside it (without it: `can't read "tracefile": no such variable`, Appendix B).

```
9      $ns flush-trace
10     close $namfile
11     close $tracefile
12     exec nam exp3.nam &
```

`flush-trace`, close the two files, and start NAM in the background (`exec nam exp3.nam &`), as in Experiment 1.

```
13     exec awk -f exp3.awk exp3.tr > exp3.graph &
```

New: `exec awk -f exp3.awk exp3.tr > exp3.graph &` runs the **AWK throughput script** on the trace file *from inside the simulation*. `>` sends the output of `awk` into the file `exp3.graph` (instead of the screen) and

`&` runs it in the background. So after `ns exp3.tcl` you only have to type `xgraph exp3.graph`.

```
14      exit 0
15  }
```

`exit 0` ends `ns`; line 15 closes the procedure.

```
17  # Create 6 nodes
18  set n0 [$ns node]
19  set n1 [$ns node]
20  set n2 [$ns node]
21  set n3 [$ns node]
22  set n4 [$ns node]
23  set n5 [$ns node]
24  set n6 [$ns node]
```

Seven nodes `n0` to `n6`. (The comment says six, but seven are created: `n0` stands outside the LAN.)

```
26  # label source and sink nodes
27  $n0 label "TCPSource"
28  $n6 label "TCPSink"
```

Labels for NAM: `n0` is called `TCPSource`, `n6` `TCPSink`. Decoration only.

```
30  # color to the flow
31  $ns color 1 red
```

`$ns color 1 red`: the packets of **flow id 1** are drawn red in NAM. The flow id is set by `class_` in line 43. NAM only; no effect on the numbers.

```
33  # color to the nodes
34  $n0 color blue
35  $n6 color orange
```

NAM colours of the nodes: the source blue, the sink orange.

```
37  $ns duplex-link $n0 $n1 3Mb 20ms DropTail
```

The **access link** from the source to the LAN: duplex, **3 Mb/s**, 20 ms, DropTail. `n1` is at the same time the LAN node that receives from `n0`.

```
38  $ns make-lan "$n1$n2$n3$n4$n5$n6" 2Mb 40ms LL Queue/DropTail Mac/802_3
```

`make-lan` builds the Ethernet: `$ns make-lan "nodes" bandwidth delay LL queue MAC`. The nodes in quotes (`n1`–`n6`) share **one channel** of **2 Mb/s** with **40 ms** delay. Every node receives a **link layer** (`LL`), an **interface queue** (`Queue/DropTail`) and the **MAC protocol** `Mac/802_3`, which is Ethernet with CSMA/CD. The `2Mb` here is the *data rate* that this experiment varies (0.5, 1, 2, 3 Mb). The whole command must stay on **one line** (Appendix B: otherwise the last arguments are lost without an error message).

```
40  $ns duplex-link-op $n0 $n1 orient right
```

NAM layout for the link `n0-n1` (to the right). No effect on the simulation.

```
41  set tcp [new Agent/TCP]
42  $ns attach-agent $n0 $tcp
43  $tcp set class_ 1
```

The **TCP sender** on `n0` (line 41 creates it, 42 attaches it). `$tcp set class_ 1` gives its packets the flow id 1: it is the field `$8` of the trace lines and selects the NAM colour of line 31.

```
45  set tcpsink [new Agent/TCPSink]
46  $ns attach-agent $n6 $tcpsink
47  $ns connect $tcp $tcpsink
```

The **TCP sink** on `n6` (it sends the acknowledgements), and `connect`, which tells the sender where to send.

```
49 set ftp [new Application/FTP]
50 $ftp attach-agent $tcp
```

The FTP source: unlimited data, attached to the TCP agent.

```
52 $ns at 1.0 "$ftpstart"
53 $ns at 5.0 "$ftpstop"
54 $ns at 5.5 "finish"
55 $ns run
```

FTP is on from 1.0 s to 5.0 s; `finish` runs at 5.5 s (the last packets are still travelling for 0.5 s, and `awk` needs the complete file); `$ns run` starts everything.

Line by line: the error model (insert after line 37)

These seven lines are *added* to `exp3.tcl` directly after the command that creates the link `n0-n1`.

```
1 # Error model on the link n0 -> n1
```

Comment: the model will act on the link from `n0` to `n1`.

```
2 set em [new ErrorModel]
```

`new ErrorModel` creates an **error model** object, stored in `em`: a module that decides for every packet passing through it whether the packet is damaged.

```
3 $em unit pkt
```

`unit pkt`: the error rate is applied **per packet** (other units are `byte` and `time`).

```
4 $em set rate_ 0.01
```

`rate_ 0.01`: every packet is discarded with probability **0.01 = 1 %**. This is the number you change (0, 0.001, 0.005, 0.01 in the manual).

```
5 $em ranvar [new RandomVariable/Uniform]
```

`ranvar`: the **random-number generator** used for the decision; a uniform random variable (equally likely values between 0 and 1) is compared with the rate.

```
6 $em drop-target [new Agent/Null]
```

`drop-target`: where the damaged packets are sent: a Null agent, which simply discards them.

```
7 $ns lossmodel $em $n0 $n1
```

`$ns lossmodel $em $n0 $n1` **inserts** the model into the link from `n0` to `n1`. It acts in this direction only, i.e. on the data packets (the acknowledgements come back on the other direction). Every lost packet makes TCP shrink its window or wait for a timeout, so the throughput falls.

Line by line: exp3.awk

This script does exactly what `exp2_throughput.awk` does (see Experiment 2); only the layout of the braces differs.

```
# gedit exp3.awk
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
```

Comment and `BEGIN`: `sSize` (bytes received so far) is 0, `startTime` starts at 5.0 (larger than any time, so the first enqueue replaces it), `stopTime` starts at 0.1 (smaller than any time, so the first receive replaces it), `Tput` is 0.

```
{
    event = $1;
    time = $2;
    size = $6;
```

Main block: for every trace line copy the three fields that are needed: `event = $1`, `time = $2`, `size = $6`.

```
    if ( event == "+")
    {
        if ( time < startTime )
        {
            startTime = time;
        }
    }
```

If the event is `+` and `time` is smaller than `startTime`, store it: `startTime` becomes the time of the **first packet sent** (here 1.0 s, when FTP starts).

```
    if ( event == "r")
    {
        if ( time > stopTime )
        {
            stopTime = time;
        }
        sSize += size;
    }
```

If the event is `r`: raise `stopTime` if this is the latest receive so far, and add the packet size to `sSize` (all packets received on all links).

```
    Tput = ( sSize /( stopTime - startTime )) * (8/1000) ;
```

`Tput = ( sSize /( stopTime - startTime )) * (8/1000)`: bytes per second × 8 / 1000 = **kb/s**.

```
    printf("%f\t %.2f\n", time , Tput );
}
END {
}
```

Print `time` and `Tput` for every line; close the main block; `END` is empty.

The script counts every receive on every link, so a packet that crosses the access link and then the LAN is counted twice: the curve shows the **aggregate throughput of the network**, which is the quantity compared in the manual (for example 1327 kb/s at the end with a 2 Mb LAN and 0 % errors).

Example of the beginning and the end of exp3.graph (from the manual)

```
$ head -3 exp3.graph ; tail -2 exp3.graph
1.000000 -0.00
1.000000 -0.00
1.020107 15.91
...
5.202373 1328.39
```

```
5.206637 1327.12
```

Change it: what the examiner may ask

Task	Change	Where
other LAN data rate	`2Mb` → `1Mb`	line 38
other LAN delay	`40ms` → `10ms`	line 38
`N` nodes in the LAN (6 to 10)	create `n7`, `n8`, … as in lines 18–24 and add them to the quoted list	lines 18–24, 38
other access link	`3Mb 20ms`	line 37
introduce an error rate	insert the seven lines of the error model; change `rate_`	after line 37
errors in the other direction	`$ns lossmodel $em $n1 $n0`	error model line 7
longer transfer	`stop` and `finish` times	lines 53, 54

> **In the exam.** Keep a copy of each result before you run again: `cp exp3.graph rate_1Mb`, then `xgraph exp3.graph rate_1Mb rate_2Mb` draws the curves of several runs in one window (Appendix C of the manual). Compare the last value of each curve.

Check yourself

1. What does `Mac/802_3` stand for in `make-lan`? (*the Ethernet MAC protocol, CSMA/CD*)
2. Which command starts `awk` automatically at the end of the simulation, and what do `>` and `&` do? (*line 13; `>` writes the output into `exp3.graph`, `&` runs it in the background*)
3. Why does the throughput hardly grow from a 2 Mb to a 3 Mb LAN? (*the LAN is no longer the bottleneck: the TCP window and the round-trip time limit the rate*)
4. In which direction does the error model drop packets, and why does that hurt TCP? (*n0→n1, the data direction; every loss makes TCP reduce its window or wait for a timeout*)
5. What is `class_ 1` used for? (*flow id in field `$8` of the trace and the NAM colour*)
6. What must you change to simulate an 8-node LAN? (*create two more nodes and add them to the list inside the quotes of `make-lan`*)

EXPERIMENT 4

Experiment 4: Congestion Window of TCP Vegas and TCP Reno

What the program does and why

Nine nodes. Two TCP senders, `n1` and `n2`, are connected to the router `n0`; `n0` is connected to the LAN node `n3`; the LAN `n3`–`n8` (512 kb/s, CSMA/CD) carries both flows. **Flow 1** is **TCP Vegas** from `n1` to `n7` (starts at 0.5 s); **flow 2** is **TCP Reno** from `n2` to `n8` (starts at 1.0 s); both use FTP and small packets of 55 bytes. NS-2 writes every change of the **congestion window** `cwnd_` of each sender into a file; AWK extracts time and window, and `xgraph` plots them.

Two ideas you need

- **Congestion window.** The sender may have at most `cwnd_` packets unacknowledged. Slow start: the window grows by one packet per acknowledgement (it doubles every round-trip time). Congestion avoidance: about one packet per round-trip time. In NS-2 the number of packets really in flight is also limited by `window_` (20 packets by default); `cwnd_` itself keeps changing.
- **Reno and Vegas.** Reno increases the window until a packet is lost and then halves it. Vegas compares the expected and the actual rate (using the round-trip delay) and keeps the difference between two small thresholds, so its window settles at a steady value without waiting for a loss.

The method

1. Create files and `finish`; `finish` also converts the two window files with AWK (lines 1–19).
2. Nodes, colours, node shapes, links and the LAN (lines 21–53).
3. Flow 1: Vegas agent on `n1`, sink on `n7`, FTP, window trace file `cwnd1.tr` (lines 55–72).
4. Flow 2: Reno agent on `n2`, sink on `n8`, FTP, window trace file `cwnd2.tr` (lines 74–89).
5. Events and run (lines 91–96); then `xgraph tcp1` and `xgraph tcp2`.

Line by line: exp4.tcl

```
1 set ns [new Simulator]
2 set namfile [open exp4.nam w]
3 $ns namtrace-all $namfile
4
5 set tracefile [open exp4.tr w]
6 $ns trace-all $tracefile
```

Simulator, NAM file `exp4.nam` (`namtrace-all`) and text trace `exp4.tr` (`trace-all`): as in the earlier experiments.

```
8 proc finish {} {
9     global ns namfile tracefile tfile1 tfile2
```

`proc finish {}` and its `global` line. This time **five** variables are needed: the two usual files `namfile`, `tracefile` and the two window files `tfile1`, `tfile2` (and `ns`). Forgetting `tfile1 tfile2` gives `can't read "tfile1": no such variable`.

```
10      $ns flush-trace
11      close $namfile
12      close $tracefile
13      close $tfile1
14      close $tfile2
```

Flush the trace and close **four** files: the NAM file, the trace file and the two window files (so that AWK sees them complete).

```
15      exec nam exp4.nam &
```

Start NAM in the background.

```
16      exec awk -f exp4.awk cwnd1.tr > tcp1 &
17      exec awk -f exp4.awk cwnd2.tr > tcp2 &
```

Two AWK runs: `awk -f exp4.awk cwnd1.tr > tcp1` converts the window file of flow 1 into the two-column file `tcp1`; line 17 does the same for flow 2 (`tcp2`). `&` runs them in the background. Plot with `xgraph tcp1` and `xgraph tcp2`.

```
18      exit 0
19  }
```

`exit 0` and the end of the procedure.

```
21  # Create the nodes
22  set n0 [$ns node]
23  set n1 [$ns node]
24  set n2 [$ns node]
25  set n3 [$ns node]
26  set n4 [$ns node]
27  set n5 [$ns node]
28  set n6 [$ns node]
29  set n7 [$ns node]
30  set n8 [$ns node]
```

Nine nodes `n0`–`n8`. `n0` is the router, `n1`, `n2` the senders, `n3`–`n8` the LAN; `n7` and `n8` are the receivers.

```
32  # color to packet flow
33  $ns color 1 Blue
34  $ns color 2 Red
```

NAM colours of the flows: flow id 1 is blue (Vegas), flow id 2 is red (Reno). The ids are assigned by `class_` in lines 67 and 84.

```
36  # set shape and color to node 7 and node 8
37  $n7 shape Box
38  $n7 color Blue
39  $n8 shape hexagon
40  $n8 color Red
```

NAM appearance: `n7` is drawn as a blue box, `n8` as a red hexagon, so the two receivers are easy to tell apart. No effect on the result.

```
42  $ns duplex-link $n1 $n0 2Mb 10ms DropTail
43  $ns duplex-link $n2 $n0 2Mb 10ms DropTail
44  $ns duplex-link $n0 $n3 1Mb 20ms DropTail
```

The access links: `n1-n0` and `n2-n0` with 2 Mb/s and 10 ms; `n0-n3` with 1 Mb/s and 20 ms (this link leads into the LAN).

```
46  # Create LAN
47  $ns make-lan "$n3$n4$n5$n6$n7$n8" 512Kb 40ms LL Queue/DropTail Mac/802_3
```

`make-lan` builds the Ethernet from `n3`–`n8`: **512 kb/s**, 40 ms, link layer `LL`, interface queue `Queue/DropTail`, MAC `Mac/802_3` (CSMA/CD). The two flows and their acknowledgements share this slow channel, so collisions and back-offs occur here. Keep the command on one line.

```
# Set Orientation
$ns duplex-link-op $n1 $n0 orient right-down
$ns duplex-link-op $n2 $n0 orient right-up
$ns duplex-link-op $n0 $n3 orient right
```

NAM layout of the three links.

```
$ns queue-limit $n0 $n3 20
```

`queue-limit $n0 $n3 20`: the buffer of the link `n0→n3` is only **20 packets** (the default is 50).

```
# set the transport protocol
set tcp1 [new Agent/TCP/Vegas]
$ns attach-agent $n1 $tcp1
```

Flow 1, sender: `new Agent/TCP/Vegas` creates a **TCP Vegas** agent (delay-based congestion control) and line 57 attaches it to `n1`.

```
set sink1 [new Agent/TCPSink]
$ns attach-agent $n7 $sink1
```

Flow 1, receiver: a `TCPSink` on `n7`; it sends the acknowledgements.

```
# set the application
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1
```

The **FTP** source of flow 1, attached to the Vegas agent (unlimited data).

```
$ns connect $tcp1 $sink1
$tcp1 set class_ 1
$tcp1 set packetSize_ 55
```

`connect` joins sender and sink. `class_ 1` is the flow id (blue). `packetSize_ 55` makes the packets of this flow only 55 bytes of payload (95 bytes with headers), so many packets and many window changes occur.

```
set tfile1 [open cwnd1.tr w]
$tcp1 attach $tfile1
$tcp1 trace cwnd_
```

Window tracing. Line 70 opens `cwnd1.tr` for writing. Line 71 `attaches` this file to the agent as its trace channel, and line 72 `trace cwnd_` tells the agent to write a line into it **every time `cwnd_` changes**: time, source node, source port, destination node, destination port, the name `cwnd_` and its new value, for example `0.94645 1 0 7 0 cwnd_ 2.000`.

```
set tcp2 [new Agent/TCP/Reno]
$ns attach-agent $n2 $tcp2
```

Flow 2, sender: `new Agent/TCP/Reno` (**TCP Reno**, loss-based congestion control), attached to `n2`.

```
set sink2 [new Agent/TCPSink]
$ns attach-agent $n8 $sink2
```

Flow 2, receiver: a `TCPSink` on `n8`.

```
set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2
```

The FTP source of flow 2, attached to the Reno agent.

```
$ns connect $tcp2 $sink2
$tcp2 set class_ 2
$tcp2 set packetSize_ 55
```

`connect` the second pair; `class_ 2` (red) and `packetSize_ 55`.

```
87 set tfile2 [open cwnd2.tr w]
88 $tcp2 attach $tfile2
89 $tcp2 trace cwnd_
```

The same window tracing for flow 2, into `cwnd2.tr`.

```
91 $ns at 0.5 "$ftp1start"
92 $ns at 1.0 "$ftp2start"
93 $ns at 5.0 "$ftp2stop"
94 $ns at 5.0 "$ftp1stop"
95 $ns at 5.5 "finish"
96 $ns run
```

Flow 1 starts at **0.5 s**, flow 2 at **1.0 s**; both stop at 5.0 s (lines 93 and 94 are two separate events at the same time); `finish` runs at 5.5 s; `$ns run` starts the simulation.

Line by line: exp4.awk

```
1 # exp4.awk
2 BEGIN {
3 }
```

Comment and an empty `BEGIN` block (nothing to prepare).

```
4 {
5     if( $6 == "cwnd_")
6     {
7         printf("%f\t %f\n", $1 , $7 );
8     }
9 }
```

For every line of a window file: `$6` is the **name of the traced variable**; the test `$6 == "cwnd_"` keeps only the window lines (useful if other variables were traced as well). `printf("%f\t %f\n", $1, $7)` prints the time (`$1`) and the window value (`$7`) as two columns, which is the format `xgraph` needs.

```
10 END {
11 }
```

Empty `END` block.

Dry run: a small window file and the output of awk -f exp4.awk (the ssthresh_ line is made up to show the filter)

```
0.00000 1 0 7 0 cwnd_ 1.000
0.94645 1 0 7 0 cwnd_ 2.000
0.94645 1 0 7 0 ssthresh_ 20
1.39290 1 0 7 0 cwnd_ 3.000

$ awk -f exp4.awk mini4.tr
0.000000^^I 1.000000
0.946450^^I 2.000000
1.392900^^I 3.000000
```

Reading the graphs of the manual

- Both curves start with **slow start**: a quick rise from 1 packet (Vegas from 0.5 s, Reno from 1.0 s). The staircase shape arises because the window only changes when an acknowledgement arrives.
- **Vegas** stops at about 22 packets and stays there: its delay measurements show that the network is full, so it keeps the window. **Reno** keeps growing linearly (30.7 packets at 5.1 s) because no packet is lost: the trace contains no `d` lines.
- A likely reason why Reno never loses a packet is that the actual sending is limited to `window_` = 20 packets per flow. At most 2 × 20 small packets are in the network, fewer than the queues can hold, so no queue overflows. `cwnd_` can therefore keep growing without any visible effect, exactly as Reno's curve does.
- Both curves drop to 1 packet after 5.0 s, when the FTP sources stop.

Change it: what the examiner may ask

Task	Change	Where
other TCP variant	`Agent/TCP/Vegas` → `Agent/TCP/Newreno` or `Agent/TCP/Reno`	lines 56, 74
other LAN speed	`512Kb` → `1Mb`	line 47
smaller queue (to see losses)	`queue-limit 20` → `5` and `$tcp1 set window_ 50`	line 53
other packet size	`packetSize_ 55` → `1000`	lines 68, 85
different start times	`at 0.5`, `at 1.0`	lines 91, 92
both curves in one window	`xgraph tcp1 tcp2`	–
a third flow	copy lines 74–89 with new names and a new `class_`	after line 89

> **Careful.** If the examiner asks for the **number of collisions**, remember that this script does not count them. Collisions are resolved by CSMA/CD with random back-off, so the TCP flows still complete without drops; compare the manual's observation that the trace contains no `d` line.

Check yourself

1. What does `$tcp1 trace cwnd_` do, and what does `$tcp1 attach $tfile1` add? (*it logs every change of `cwnd_`; the attach gives it the file to write to*)
2. Why does `finish` list `tfile1 tfile2` in `global`? (*it closes these two file handles, which were created outside the procedure*)
3. What does `$6 == "cwnd_"` test in `exp4.awk`? (*that the line is a window line*)
4. Why is the Vegas curve flat but the Reno curve still rising? (*Vegas holds its window when it detects queueing delay; Reno only reacts to loss and there is none*)
5. What is the difference between slow start and congestion avoidance? (*exponential versus about one packet per round trip*)
6. Which line sets the queue of the link `n0→n3`? (*line 53*)

EXPERIMENT 5

Experiment 5: Bit Stuffing in the Data-Link Layer

What the program does and why

The data-link layer cuts the stream of bits into **frames**. A common way to mark where a frame starts and ends is a special **flag**; HDLC uses the bit pattern `01111110` (a 0, six 1s, a 0). If the *data* itself contained six 1s in a row, the receiver would think the frame had ended. **Bit stuffing** makes sure that six 1s in a row can never occur in the data: the sender inserts a `0` after every run of **five 1s**; the receiver removes the `0` that follows five 1s. The program does both and shows that the original data comes back.

The method

- **Sender:** go through the bits from left to right and count the consecutive 1s. Copy every bit; when the count reaches 5, also copy a `0` and start counting again. A `0` in the data resets the count.
- **Receiver:** go through the received bits and count the consecutive 1s in the same way; when the count reaches 5, the *next* bit is the stuffed `0` and must be skipped.
- **Overhead:** at worst (all 1s) one extra bit for every five data bits (20 %).

Line by line: exp5.py

```
1 def bit_stuffing(data):
```

`def bit_stuffing(data):` **defines a function** with one parameter, `data`: a *string* of the characters `0` and `1` (for example `"1011111101"`). The function returns the stuffed string.

```
2     stuffed_data = ""
3     count = 0
```

`stuffed_data = ""` is the result built so far (an empty string). `count = 0` counts the **consecutive 1s** just seen.

```
4     for bit in data:
5         stuffed_data += bit
```

`for bit in data:` visits the characters of the string one after the other; `bit` is `"1"` or `"0"`. Line 5 first **copies the bit** to the result (`+=` appends to a string).

```
6         if bit == '1':
7             count += 1
```

`if bit == '1':` compares with the *character* `'1'` (the data is text, not a number). If the bit is 1, `count += 1` counts it as one more 1 in the run.

```
8             if count == 5:
9                 stuffed_data += '0'
10                count = 0
```

When the count reaches **5**, a `0` is appended (this is the **stuffed bit**) and `count = 0` starts a new run. The inserted `0` itself ends the run, which is why the counter is reset to 0 and not to 1.

```
11          else:
12              count = 0
```

`else:` belongs to the `if bit == '1'` of line 6: the bit is a `0`, which **breaks the run**, so `count = 0`.

```
13      return stuffed_data
```

`return stuffed_data` hands the result back to the caller.

```
15  def bit_destuffing(stuffed_data):
16      destuffed_data = ""
17      count = 0
18      skip_next = False
```

`bit_destuffing` is the receiver. `destuffed_data` is the result, `count` again counts consecutive 1s, and `skip_next = False` is a **flag**: when it is `True`, the next bit is a stuffed `0` that must not be copied.

```
19      for bit in stuffed_data:
20          if skip_next:
21              skip_next = False
22              continue
```

Loop over the received bits. If `skip_next` is set, this bit is the stuffed 0: clear the flag (`skip_next = False`) and `continue`, which jumps straight to the next bit without executing the rest of the loop body (the stuffed bit is not copied).

```
23          destuffed_data += bit
24          if bit == '1':
25              count += 1
26              if count == 5:
27                  skip_next = True
28                  count = 0
```

Otherwise the bit is copied (line 23). If it is a 1, count it (24–25); when the count reaches 5, the **next** bit is a stuffed 0, so `skip_next = True`, and `count = 0` (26–28).

```
29          else:
30              count = 0
```

A `0` breaks the run: `count = 0`.

```
31      return destuffed_data
```

Return the destuffed data.

```
33  data = input("Enter the input data :")
34  print("Original data :", data)
```

The main program starts here (no indentation). `input(...)` reads the data as a string; line 34 prints it. `print("text", value)` puts a blank between its arguments.

```
35  stuffed = bit_stuffing(data)
36  print("Bit Stuffed data :", stuffed)
```

Calls the sender function and prints the stuffed data.

```
37  destuffed = bit_destuffing(stuffed)
38  print("Bit Destuffed data :", destuffed)
```

Calls the receiver function on the stuffed data and prints the result, which must equal the original.

Dry run for 1011111101

The program was traced line by line for the input of the manual. The six 1s of the data (positions 2 to 7, counting from 0) are interrupted by the stuffed `0` after the fifth.

Real trace of bit_stuffing and bit_destuffing for 1011111101

```
SENDER  bit_stuffing("1011111101")
 i  bit  count(before)  what happens                          stuffed_data
 0   1        0        1 -> count = 1                         1
 1   0        1        0 -> count = 0                         10
 2   1        0        1 -> count = 1                         101
 3   1        1        1 -> count = 2                         1011
 4   1        2        1 -> count = 3                         10111
 5   1        3        1 -> count = 4                         101111
 6   1        4        count reaches 5 -> add '0', count = 0  10111110
 7   1        0        1 -> count = 1                         101111101
 8   0        1        0 -> count = 0                         1011111010
 9   1        0        1 -> count = 1                         10111110101
result: 10111110101   (10 bits became 11 bits)

RECEIVER  bit_destuffing("10111110101")
 i  bit  skip_next  what happens                          destuffed_data
 0   1     False    1 -> count = 1                              1
 1   0     False    0 -> count = 0                              10
 2   1     False    1 -> count = 1                              101
 3   1     False    1 -> count = 2                              1011
 4   1     False    1 -> count = 3                              10111
 5   1     False    1 -> count = 4                              101111
 6   1     False    count reaches 5 -> skip_next = True         1011111
 7   0     True     skip_next was True -> ignore this bit       1011111
 8   1     False    1 -> count = 1                              10111111
 9   0     False    0 -> count = 0                              101111110
10   1     False    1 -> count = 1                              1011111101
result: 1011111101   = the original data
```

The result `10111110101` is the one in the manual. Note that the stuffed stream contains at most five 1s in a row, so `01111110` can never appear inside the data.

Special cases

Input	Stuffed	Why
`11111`	`111110`	a run of five at the very end is stuffed too (the receiver cannot know what follows)
`1111111111`	`111110111110`	ten 1s: two stuffed bits, the 20 % worst case
`0000`	`0000`	no run of 1s: nothing is inserted
`111110`	`1111100`	the 0 of the data follows a stuffed 0

Change it: what the examiner may ask

Task	Change	Where
stuff after four 1s instead of five	`5` → `4` (both functions)	lines 8, 26
insert a 1 after five 0s	swap the roles of `'1'` and `'0'`	lines 6, 9, 24
print the frame with flags	`frame = "01111110" + stuffed + "01111110"`	after line 35
check the input	`if set(data) <= {'0','1'}` before line 35	line 34
count the stuffed bits	`len(stuffed) - len(data)`	after line 35

> **In the exam.** Byte stuffing is the character-oriented relative: a flag byte or escape byte inside the data is preceded by an escape byte (PPP). If you are asked to *explain*, give three sentences: why a flag is needed, why the data can contain the flag, and how stuffing prevents it.

Check yourself

1. Why is the counter reset to 0 (and not 1) after the stuffed bit is added? (*the stuffed 0 breaks the run of 1s*)
2. What does `continue` do on line 22? (*skips the rest of the loop body for this bit*)
3. Why is a `0` stuffed even when the five 1s are the last bits of the data? (*the receiver cannot know whether more bits follow, so the rule is applied unconditionally*)
4. What is the worst-case overhead, and for which input? (*20 %, all 1s*)
5. Stuff `01111110` by hand. (*six 1s: `011111010`; the stuffed 0 follows the fifth 1*)
6. Which line decides that the *next* bit of the receiver is ignored? (*line 27, `skip_next = True`*)

EXPERIMENT 6

Experiment 6: Error Detection Using CRC (CRC-CCITT)

What the program does and why

A **cyclic redundancy check** adds a few check bits to the data so that the receiver can detect transmission errors. Sender and receiver agree on a **generator polynomial** G(x). The sender appends zeros to the data, divides by G using **modulo-2 arithmetic** (division with XOR, no carries) and appends the *remainder* to the data. The resulting codeword is exactly divisible by G. The receiver divides what it received by G: **remainder 0 means no error detected; a non-zero remainder means the frame is corrupt**. The program lets you flip one bit to see the detection.

The method in five steps

1. Write the data as a list of bits; let `m` be the length of the generator (its degree plus 1).
2. Append `m - 1` zeros to a copy of the data.
3. **Divide:** for every position `i` of the data, if the bit there is 1, XOR the generator into the bits starting at `i`.
4. The last `m - 1` bits that remain are the **remainder**; the codeword is the data followed by the remainder.
5. The receiver repeats step 3 on the received bits; if every bit is 0 afterwards, no error.

> **Note.** The lines of the program use the generator `1011` ($x^3 + x + 1$) so that the division can be done by hand. For **CRC-CCITT** only line 1 changes: `div = [1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1]` ($x^{16} + x^{12} + x^5 + 1$, 17 bits, hexadecimal 0x11021), and the data must have at least 17 bits.

Line by line: exp6.py

```
1 div = [1, 0, 1, 1]
2 cdata = []
```

`div = [1, 0, 1, 1]` is the **generator polynomial** as a list of bits: $1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1$. `cdata = []` creates an empty list that will hold the data with the appended zeros.

```
3 n = int(input("Enter the number of bits :"))
```

`input(...)` reads the number of data bits as a string and `int(...)` converts it to a number `n`.

```
4 data = [int(input(f"Enter bit {i + 1}: ")) for i in range(n)]
```

A **list comprehension** reads `n` bits: for each `i` from 0 to `n-1` it asks `Enter bit i+1:` (the f-string puts the value of `i + 1` into the text) and converts the answer to `int`. The result is the list `data`, for example `[1, 0, 0, 1]`.

```
5 m = len(div)
```

`m = len(div)`: the length of the generator, 4 here. The number of check bits is `m - 1`.

```
7  if n < m:
8      print("Invalid data ")
9  else:
```

`if n < m:` the data is shorter than the generator, so the division makes no sense: print `Invalid data`. Otherwise (`else:`, line 9) the program continues; everything below is indented one level deeper.

```
10     cdata = data.copy()
11     for i in range(m - 1):
12         cdata.append(0)
```

`data.copy()` makes an **independent copy**, so that adding zeros to `cdata` does not change `data`. The loop appends `m - 1` zeros: for `1001` the list becomes `1001000`. These zeros make room for the remainder.

```
13     print("Initial Codeword :", cdata)
```

Prints the list; the manual calls it the *initial codeword*.

```
14     for i in range(n):
15         if cdata[i] == 1:
16             for j in range(m):
17                 cdata[i + j] ^= div[j]
```

The **division**. For each of the first `n` positions: if the bit `cdata[i]` is 1, XOR the whole generator into `cdata` starting at `i` (`cdata[i + j] ^= div[j]` for `j = 0 … m-1`). XOR with the generator's first bit (1) clears `cdata[i]`, so the leading 1 disappears, exactly like a step of long division in which you subtract the divisor. If `cdata[i]` is 0, nothing is done. After the loop, `cdata[0..n-1]` are 0 and the last `m - 1` bits are the **remainder**.

```
19     for i in range(n, n + m - 1):
20         data.append(cdata[i])
21     print("Codeword to be transmitted is:", data)
```

Append the remainder to `data` (`cdata[n]` to `cdata[n+m-2]`) and print: `data` is now the **codeword to be transmitted** = original data + CRC.

```
22     ch = input("Introduce Error - Y -> YES N->NO:")
23     if ch.upper() == 'Y':
```

Asks whether to introduce an error. `ch.upper()` converts the answer to capital letters, so `y` and `Y` both work.

```
24         p = int(input("Enter the position to change bit :"))
25         if 1 <= p <= len(data):
26             data[p-1] ^= 1
```

Reads the **position** `p` (counting from 1). If it lies inside the codeword (`1 <= p <= len(data)`), `data[p-1] ^= 1` **flips** that bit (XOR with 1 changes 0 to 1 and 1 to 0). The index is `p-1` because Python counts from 0.

```
27         else:
28             print("Invalid Position !")
```

Otherwise the message `Invalid Position !` is printed and nothing is changed.

```
29     print("The message received is:", data)
```

Prints the message as received by the receiver (with the error, if one was introduced).

```
30     for i in range(n):
31         if data[i] == 1:
32             for j in range(m):
33                 data[i + j] ^= div[j]
```

The **receiver divides** the received codeword by the generator, with the same loop as lines 14–17 but working directly on `data`. A valid codeword leaves all zeros.

```
34      flag = True
35      for i in data:
36          if i == 1:
37              flag = False
38              break
```

`flag = True` assumes that no error occurred; the loop looks at all bits and, as soon as it finds a 1 (a non-zero remainder), sets `flag = False` and leaves the loop with `break`.

```
39      if (flag):
40          print("No Error \nSuccessfully Transferred")
41      else:
42          print("Error !")
```

If `flag` is still `True`: `No Error / Successfully Transferred`; otherwise `Error !`.

Dry run with the data 1001 and the generator 1011

Real trace of the two division loops (sender, receiver without error, receiver with bit 1 flipped)

```
SENDER   data = 1001, div = 1011, n = 4, m = 4
initial codeword (data + 3 zeros): 1001000
i=0: cdata[0] = 1 -> XOR 1011 at positions 0..3:   1001000  ->  0010000
i=1: cdata[1] = 0 -> nothing to do:                 0010000
i=2: cdata[2] = 1 -> XOR 1011 at positions 2..5:   0010000  ->  0000110
i=3: cdata[3] = 0 -> nothing to do:                 0000110
remainder = last 3 bits = 110   ->   codeword = 1001110

RECEIVER, no error:  received = 1001110
i=0: bit = 1 -> XOR:   1001110  ->  0010110
i=1: bit = 0 -> nothing:                0010110
i=2: bit = 1 -> XOR:   0010110  ->  0000000
i=3: bit = 0 -> nothing:                0000000
any 1 left?  no -> No Error

RECEIVER, bit 1 flipped:  received = 0001110
i=0: bit = 0 -> nothing:                0001110
i=1: bit = 0 -> nothing:                0001110
i=2: bit = 0 -> nothing:                0001110
i=3: bit = 1 -> XOR:   0001110  ->  0000101
any 1 left?  yes (0000101) -> Error!
```

This is the worked example of the manual: the CRC of `1001` is `110`, the codeword is `1001110`; when the first bit is flipped the remainder is `101` and the error is detected.

Is the program really CRC-CCITT?

Yes. With the 17-bit generator the same loops give the standard result: applied to the ASCII text `123456789` the program's algorithm produces **0x31C3**, which is the published check value of CRC-16/XMODEM (CRC-CCITT with start value 0, no bit reflection, no final XOR) and equals Python's `binascii.crc_hqx(b"123456789", 0)`. The manual's 20-bit example `10110100111001011001` gets the CRC `0001010111101110`, the 16 bits printed after the data in the manual. (Other "CCITT" variants start the register with 0xFFFF and give different values; the lab uses the textbook version.)

Why the check works

- The transmitted codeword is a multiple of G(x). An error adds an error polynomial E(x); the receiver sees a multiple of G plus E, so the remainder is non-zero unless E itself is a multiple of G.
- Since $x^{16} + x^{12} + x^5 + 1 = (x + 1) \cdot (\ldots)$, it detects every error with an *odd* number of flipped

bits; it also detects all single-bit and double-bit errors and all bursts of 16 bits or fewer, as stated in the manual.

Change it: what the examiner may ask

Task	Change	Where
another generator, e.g. `x^4+x+1`	`div = [1,0,0,1,1]`	line 1
CRC-CCITT (16 bits)	the 17-bit list given above; at least 17 data bits	line 1
CRC-16 (IBM)	`x^16+x^15+x^2+1`: `[1,1,0,0,0,0,0,0,0,0,0,0,0,1,0,1]`	line 1
flip two bits (burst)	also `data[p] ^= 1` after line 26	line 26
show the remainder	`print(data[n:])` after the sender loop	after line 17
read the data as a string	`data = [int(c) for c in input()]`	line 4

In the exam. Typical oral questions: *why do we append zeros?* (to make room for the remainder, so that data followed by the remainder is divisible by G); *what is modulo-2 arithmetic?* (addition and subtraction are both XOR, no carries); *how many check bits for a generator with* `m` *bits?* (m − 1).

Check yourself

1. Do the sender's division by hand for the data `1101` and the generator `1011`: append zeros, XOR, find the remainder. (*1101000 → 0110000 → 0011100 → 0001010 → 0000001: remainder* `001`*, codeword* `1101001`)
2. Why does line 10 use `data.copy()` and not `cdata = data`? (*with* `=` *both names would refer to the same list*)
3. What does `^=` do on line 17 and on line 26? (*XOR assignment: in the division it XORs the generator in; on line 26 it flips a bit*)
4. What does the receiver conclude when a 1 is left after the division? (*the codeword was corrupted*)
5. Why must the data have at least `m` bits in this program? (*the check* `n < m` *on line 7; otherwise there is nothing to divide*)
6. Which errors can a CRC miss? (*error patterns that are themselves multiples of* $G(x)$)

EXPERIMENT 7

Experiment 7: Frame Sorting Technique Used in Buffers

What the program does and why

A long message is cut into pieces and every piece travels in a **frame** that carries a **sequence number**. On a real network the frames may arrive in the wrong order (different paths, queues, retransmissions). The receiver therefore keeps the frames in a **buffer**, sorts them by sequence number and only then hands the message to the upper layer. The program imitates all three steps: it **fragments** a message into frames of `DATA_SZ` = 3 characters, **shuffles** them to imitate arrival in the wrong order, and **sorts** them back with **bubble sort**.

The method

1. Cut the message into pieces of 3 characters (the last piece may be shorter) and number them 1, 2, 3, ... : a frame is the pair `(number, piece)`.
2. Shuffle: for every position `i`, swap `frames[i]` with a randomly chosen frame.
3. Bubble sort by number: compare neighbours, swap them if the left one has the larger number; repeat. After pass *i* the *i* largest numbers are in their final places.
4. Join the pieces in the sorted order to get the message back.

Line by line: exp7.py

```
1 import random
2 DATA_SZ = 3
```

`import random` makes the module `random` available (needed for the shuffle). `DATA_SZ = 3` is a **constant** (written in capitals by convention): the number of characters in one frame. Change it here and the whole program adapts.

```
4 def bubb_sort(arr):
5     n = len(arr)
```

`def bubb_sort(arr):` defines the sorting function; `arr` is a list of frames. `n = len(arr)` is the number of frames.

```
6     for i in range(n):
7         for j in range(n - i - 1):
```

The **outer loop** `for i in range(n)` makes up to `n` passes. The **inner loop** `for j in range(n - i - 1)` walks along the list comparing the neighbours at `j` and `j + 1`. The range shrinks by one in every pass, because after pass `i` the last `i` items are already in their final places (the largest numbers have "bubbled" to the end).

```
8             if arr[j] > arr[j + 1]:
9                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

`arr[j] > arr[j + 1]` compares two frames. Frames are **tuples** `(number, piece)`, and tuples are compared item by item, so the number decides (it is unique). If the left frame is larger, the two are **swapped** with the tuple assignment `arr[j], arr[j + 1] = arr[j + 1], arr[j]` (Python evaluates the right side first, so no temporary variable is needed). The function changes the list `arr` itself and returns nothing.

```
11 msg = input("Enter the message:")
```

`input(...)` reads the message as a string.

```
12 msg_chunks = [msg[i:i + DATA_SZ] for i in range(0, len(msg), DATA_SZ)]
```

A **list comprehension with slicing**: `range(0, len(msg), DATA_SZ)` gives the start positions 0, 3, 6, ...; `msg[i:i + DATA_SZ]` is the piece from `i` up to (not including) `i + 3`. For `NMAMIT NITTE` this gives `['NMA', 'MIT', ' NI', 'TTE']` (the blank belongs to the third piece). A message whose length is not a multiple of 3 ends with a shorter piece.

```
13 frames = list(enumerate(msg_chunks, start=1))
14 print("Fragmented frames:", frames)
```

`enumerate(msg_chunks, start=1)` pairs every piece with a counter that starts at **1**; `list(...)` turns the result into a list: `[(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]`. Line 14 prints these fragmented frames.

```
15 for i in range(len(frames)):
16     j = random.randint(0, len(frames) - 1)
17     frames[i], frames[j] = frames[j], frames[i]
```

The **shuffle**: for every position `i`, `random.randint(0, len(frames) - 1)` chooses a random position `j` (both ends are possible) and the two frames are swapped. This simple method is enough to produce a random order; by chance it may leave the order unchanged (more likely with few frames), which is why the order differs from run to run.

```
18 print("Shuffled frames:", frames)
```

Prints the shuffled frames: these are the frames as they "arrive".

```
19 bubb_sort(frames)
20 print("Sorted frames:", frames)
```

Calls the sort and prints the sorted frames.

```
21 print("Sorted message:" + "".join(x[1] for x in frames))
```

`x[1]` is the piece of a frame; `(x[1] for x in frames)` is a **generator expression** that produces the pieces one by one; `"".join(...)` glues them together with nothing between them. The result is the original message.

Dry run of the bubble sort

For the received order 3 5 1 4 2 the sort works as follows (every comparison of the real algorithm is listed):

Bubble sort of five frames, one line per comparison (real trace)

```
received order: 3 5 1 4 2
pass 1  j=0  compare 3 and 5  -> keep   list: 3 5 1 4 2
pass 1  j=1  compare 5 and 1  -> swap   list: 3 1 5 4 2
pass 1  j=2  compare 5 and 4  -> swap   list: 3 1 4 5 2
pass 1  j=3  compare 5 and 2  -> swap   list: 3 1 4 2 5
pass 2  j=0  compare 3 and 1  -> swap   list: 1 3 4 2 5
pass 2  j=1  compare 3 and 4  -> keep   list: 1 3 4 2 5
pass 2  j=2  compare 4 and 2  -> swap   list: 1 3 2 4 5
pass 3  j=0  compare 1 and 3  -> keep   list: 1 3 2 4 5
pass 3  j=1  compare 3 and 2  -> swap   list: 1 2 3 4 5
pass 4  j=0  compare 1 and 2  -> keep   list: 1 2 3 4 5
comparisons = 10  (n(n-1)/2 = 10),  swaps = 6
```

With $n = 5$ frames there are always $10 = n(n-1)/2$ comparisons, because the program carries out every pass whatever the order; the number of swaps (6 here) equals the number of pairs that

are in the wrong order (*inversions*). The manual's table for the order 2 3 1 4 is the same procedure with 6 comparisons and 2 swaps.

Change it: what the examiner may ask

Task	Change	Where
frames of 5 characters	`DATA_SZ = 5`	line 2
sort in descending order	`arr[j] < arr[j + 1]`	line 8
stop early when sorted	add a flag `swapped`; `break` after a pass without swap	lines 6–9
count the swaps	add a counter in the `if` of line 8	line 9
use Python's own sort	`frames.sort()` instead of line 19	line 19
a fixed arrival order	replace lines 15–17 by e.g. `frames = [frames[2], frames[0], frames[1]]`	lines 15–17
frames with a header (sender/receiver)	use triples `(number, src, piece)`; the sort still works on the number	lines 13, 21

In the exam. If asked for the **complexity** say: bubble sort makes $n(n-1)/2$ comparisons, i.e. it grows with the square of the number of frames, which is fine for the small buffers of the experiment. The role of the **sequence number**: restore the order, detect missing frames, discard duplicates.

Check yourself

1. What is the value of `msg_chunks` for `HELLO` and `DATA_SZ = 3`? (`['HEL', 'LO']`)
2. Why does the inner loop use `n - i - 1`? (*the last i items are already in place*)
3. What does `enumerate(msg_chunks, start=1)` return for that list? (`[(1, 'HEL'), (2, 'LO')]`)
4. How can the message come back correctly although the pieces are shuffled? (*every piece keeps its sequence number, and the sort orders by it*)
5. How many comparisons does the sort make for 6 frames? (*15*)
6. Why do two runs of the program show different "Shuffled frames"? (*the positions are chosen randomly*)

EXPERIMENT 8

Experiment 8: Client-Server File Transfer Using TCP Sockets

What the programs do and why

The **server** waits for a client, receives a **file name**, and sends back the **contents** of that file (or the text `File not found`). The **client** connects, sends the file name, collects everything the server sends, shows it and saves it as `received_<name>`. Both programs use **TCP** (`SOCK_STREAM`): a reliable, connection-oriented *byte stream*. The two programs must be started in this order: **server first**, then client.

The conversation (the application protocol)

1. The client connects to (127.0.0.1, 8888).
2. The client sends the file name as bytes.
3. The server answers with the bytes of the file, or with `File not found`.
4. The server **closes** the connection. The closing tells the client that the answer is complete.

Two sockets on the server

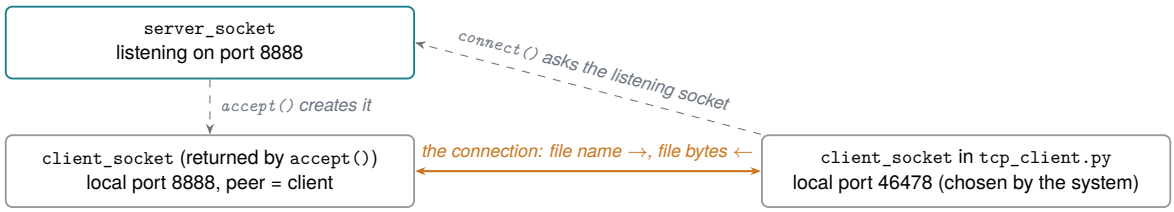


Figure A3. The listening socket only accepts new connections; every accepted connection gets a socket of its own. The client port (here 46478) is different in every run.

Line by line: tcp_server.py

```
1 # tcp_server.py - sends the requested file to the client
2 import socket
3 import os
```

Line 1 is a comment. `import socket` loads the module with the network functions; `import os` the module with operating-system helpers (here only `os.path.isfile`).

```
5 HOST = "127.0.0.1" # this computer only (localhost)
6 PORT = 8888 # port number the server listens on
7 BUFFER_SIZE = 1024 # maximum bytes received by one recv()
```

Three **constants** (capital letters by convention). `HOST = "127.0.0.1"` is the *loopback* address: only programs on this computer can connect (it is the same as `localhost`; use `"0.0.0.0"` to accept connections from other computers too). `PORT = 8888`: the port number; numbers up to 1023 are reserved for system services, so ordinary programs use higher ones. `BUFFER_SIZE = 1024`: the largest number of bytes one `recv()` will return.

```
9  # create a TCP socket, bind it to (HOST, PORT) and start listening
10 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Creates the **socket**: `socket.socket(family, type)`. `AF_INET` means the IPv4 address family, `SOCK_STREAM` means a TCP stream socket. The result `server_socket` is an object with the methods `bind`, `listen`, `accept` ... It has no address and no connection yet.

```
11 server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

`setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)` switches on the option **SO_REUSEADDR** (the `1`) at the socket level. After a server has been stopped, the operating system keeps its old connections in the TIME_WAIT state for a while; without this option a restarted server fails at `bind()` with `OSError: [Errno 98] Address already in use` (shown below). This line lets you restart the server at once. (On Windows the option behaves differently, so there stop the old server instead of relying on it.)

```
12 server_socket.bind((HOST, PORT))
```

`bind((HOST, PORT))` attaches the socket to the local address. There are **two pairs of parentheses**: `bind` takes *one* argument, the tuple `(host, port)`. Only after `bind` can clients reach the program under this port. If the port is taken: `Address already in use`.

```
13 server_socket.listen(1)
```

`listen(1)` turns the socket into a **listening socket**: it will accept incoming connections. The argument (the *backlog*) is the number of connections that may wait, not yet accepted, before new ones are refused; 1 is enough for the lab.

```
14 print("Server is listening on", HOST, "port", PORT)
```

Prints a message so that you know the server is ready. `print` separates its arguments by a blank.

```
16 try:
17     while True:
```

`try:` starts a block whose `except` and `finally` (lines 34–37) deal with Ctrl+C. `while True:` is an **infinite loop**: the server serves one client after the other until it is stopped (an *iterative* server).

```
18         client_socket, client_address = server_socket.accept()
```

`accept()` **waits** (blocks) until a client connects. It returns a pair: a **new socket** `client_socket` for the conversation with this client, and the client's address `client_address`, a tuple such as `('127.0.0.1', 46478)` (the port is chosen by the client's operating system). The listening socket stays free for the next client.

```
19         with client_socket:
20             print("Accepted connection from", client_address)
```

`with client_socket:` guarantees that the client's socket is **closed automatically** when the indented block ends, even if an error occurs. Line 20 prints who connected.

```
22             # receive the file name sent by the client
23             file_name = client_socket.recv(BUFFER_SIZE).decode()
24             print("Received file name =", file_name)
```

`recv(BUFFER_SIZE)` **waits** until the client has sent something and returns *up to* 1024 **bytes**. `.decode()` turns the bytes into a string (UTF-8), which is the file name. A short file name arrives in one piece; TCP itself gives no guarantee about how data is cut into pieces.

```
26             # send the contents if the file exists, otherwise an error text
27             if os.path.isfile(file_name):
```

`os.path.isfile(file_name)` is `True` if a regular file with this name exists (relative to the folder in which the server was started). `isfile` is better than `exists`, which is also true for folders.

```
28                 with open(file_name, "rb") as file:
29                     client_socket.sendall(file.read())
```

`open(file_name, "rb")`: open for reading in **binary** mode, so that any kind of file (text, image, PDF) is read byte for byte. `file.read()` returns all the bytes, and `sendall(...)` sends **all** of them: unlike `send`, it repeats until everything has been handed to the network or an error occurs. The `with` closes the file.

```
30                 print("File sent")
```

Reports on the server screen that the file has been sent.

```
31             else:
32                 client_socket.sendall(b"File not found")
33                 print("File not found")
```

`else:` the file does not exist. `b"File not found"` is a **bytes literal** (the `b` in front): sockets send bytes, not strings. The client recognises this text. The server prints the same message.

```
34 except KeyboardInterrupt:
35     print("\nServer stopped")
```

`except KeyboardInterrupt:` is executed when you press **Ctrl+C**; instead of an ugly error message the server prints `Server stopped`. (`\n` starts a new line after the `^C` shown by the terminal.)

```
36 finally:
37     server_socket.close()
```

`finally:` is executed in every case, also after Ctrl+C or an error: `server_socket.close()` releases the port.

The error that SO_REUSEADDR (line 11) prevents (real output; server without that line, restarted at once)

```
$ python3 tcp_server.py          # started again a second after the first run was stopped
Traceback (most recent call last):
  File "tcp_server.py", line 6, in <module>
    server_socket.bind(server_address)
OSError: [Errno 98] Address already in use
```

Line by line: tcp_client.py

```
1 # tcp_client.py - requests a file from the server and saves it
2 import socket
3 import sys
4 import os
```

Comment and three imports: `socket` for the network, `sys` for `sys.exit`, `os` for `os.path.basename`.

```
6 SERVER_ADDRESS = ("127.0.0.1", 8888) # (IP address, port) of the server
7 BUFFER_SIZE = 1024
```

`SERVER_ADDRESS = ("127.0.0.1", 8888)` is the **address of the server** as a tuple (IP address, port); it must be the same as in the server. The client needs no port of its own: the operating system assigns a free one when it connects.

```
9 # create a TCP socket and connect to the server
10 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Creates a **TCP socket**, exactly like the server's (`AF_INET`, `SOCK_STREAM`).

```
try:
    client_socket.connect(SERVER_ADDRESS)
except ConnectionRefusedError:
    sys.exit("Could not connect to the server. Is tcp_server.py running?")
```

`connect(SERVER_ADDRESS)` starts the **TCP handshake** with the server and waits until the connection is established. If no server is listening, Python raises `ConnectionRefusedError`; the `except` catches it, `sys.exit("...")` prints the message and ends the program. Without `try/except` you would see a traceback.

```
file_name = input("Enter the file name: ").strip()
if file_name == "":
    client_socket.close()
    sys.exit("No file name entered.")
```

`input(...)` reads the file name; `.strip()` removes blanks at the beginning and end. An empty name would make a **deadlock** (the server waits for data, the client waits for the answer), so the client closes the socket and stops.

```
# send the file name, then read until the server closes the connection
client_socket.sendall(file_name.encode())
```

`file_name.encode()` converts the string to **bytes** (UTF-8) and `sendall` sends them to the server.

```
file_contents = b""
```

`file_contents = b""` is an empty **bytes** value that will collect the answer.

```
while True:
    chunk = client_socket.recv(BUFFER_SIZE)
    if not chunk: # empty bytes: the server has closed
        break
    file_contents += chunk
```

The **receive loop**. `recv(BUFFER_SIZE)` returns up to 1024 bytes. If it returns an **empty bytes object**, the server has closed the connection, so the answer is complete and the loop ends with `break` (`not chunk` is `True` for empty bytes). Otherwise the piece is appended with `+=`. A file of 2592 bytes arrives as 1024 + 1024 + 544 bytes, then the empty result.

```
client_socket.close()
```

The conversation is over: close the socket.

```
if file_contents == b"File not found":
    print("File not found on the server.")
```

Compares **bytes with bytes**: if the whole answer is `b"File not found"`, print that the file is not on the server. (`==` between a string and a bytes object would never be true.)

```
else:
    print("File contents:")
    print(file_contents.decode(errors="replace"))
```

Otherwise print the contents: `file_contents.decode(errors="replace")` converts the bytes to text for the screen; bytes that are not valid UTF-8 (for example in a picture) are replaced by a replacement character instead of causing an error.

```
    save_name = "received_" + os.path.basename(file_name)
    with open(save_name, "wb") as file:
        file.write(file_contents)
    print("File received and saved as '" + save_name + "'")
```

`os.path.basename(file_name)` keeps only the last part of a path (`docs/a.txt` → `a.txt`), so the copy `received_a.txt` is created in the current folder. `open(save_name, "wb")` writes the **bytes** unchanged (`wb` = write binary), and the last line reports the name of the new file.

Dry run: what the variables contain

Step	`sample.txt` (109 bytes)	`big.txt` (2592 bytes)
server `recv` (line 23)	`file_name = 'sample.txt'`	`'big.txt'`
`isfile` (line 27)	`True`	`True`
`file.read()`	109 bytes	2592 bytes
client `recv` calls (line 25)	109, then 0 (empty)	1024, 1024, 544, then 0
`file_contents` at the end	109 bytes	2592 bytes
result	copy identical (`cmp`)	copy identical (`cmp`)

For `missing.txt`: `isfile` is `False`, the server sends the 14 bytes `File not found`, the client receives them in one `recv` and prints `File not found on the server.` without creating a file.

Why this version differs from the sample program of the lab manual

The sample `tcp_client.py` and `tcp_server.py` of the college manual work for short files that exist, but they contain three real errors, which were reproduced here. This is why the programs of Experiment 8 are written differently.

Real behaviour of the lab-manual client (Python 3.12)

```
$ python3 tcp_client.py          # lab-manual client; the file missing.txt does not exist on
    the server
Enter the file name: missing.txt
File Contents :
 File not found
File received and saved as 'received_missing.txt'
$ ls received_missing.txt; cat received_missing.txt
received_missing.txt
File not found

$ wc -c big.txt received_big.txt        # lab-manual client, file of 2592 bytes
2592 big.txt
1024 received_big.txt
3616 total
```

Problem	Lab-manual version	This version
missing file is never reported	`recv(1024).decode()` gives a *string*, but it is compared with `b'File not found'` (*bytes*): never equal	compare bytes with bytes (line 31)
long files are cut at 1024 bytes	one `recv(1024)`	receive loop until `b""` (lines 23–28)
data may be sent only partly	`send()`	`sendall()`
binary files break	text mode `'w'` and `decode()`	`"rb"` / `"wb"` and bytes
server cannot be restarted at once	no `SO_REUSEADDR`	line 11
no message if the server is down; Ctrl+C gives a traceback	—	lines 11–14 (client), 34–37 (server)

A further point of the lab manual: its *Algorithm* is fine for TCP (socket, connect, send, recv; socket,

bind, listen, accept, recv, send), but the same list was copied into Experiment 6 for UDP, where it is wrong (see the next chapter).

For a real file server (beyond the lab)

The status text `File not found` could be confused with a file that contains exactly that text. A real protocol first sends a **status and the length** of the data (for example one status byte and 8 bytes of length), so that the client knows what is coming and when it is complete without relying on the close of the connection. The server above also serves *any* path that the client names (even `../secret.txt`); a real server restricts requests to one folder. And to serve several clients at the same time it would use threads or `select`.

Change it: what the examiner may ask

Task	Change	Where
other port	`PORT`; the same number in `SERVER_ADDRESS`	server line 6, client line 6
server on another computer	server `HOST = "0.0.0.0"`, client `SERVER_ADDRESS = ("192.168.1.5", 8888)`	server 5, client 6
read in smaller pieces	`BUFFER_SIZE = 16` (the loop still works)	line 7
send the file in chunks	`while chunk := file.read(1024): client_socket.sendall(chunk)`	server 29
show the size of the file	`print(len(file_contents), "bytes")`	after client line 29
serve only one request	remove `while True:` (and indent less)	server 17
send the answer in capital letters	`file.read().upper()`	server 29

> **In the exam.** Typical oral questions: the **order of the calls** (server: socket, bind, listen, accept, recv, send, close; client: socket, connect, send, recv, close); why `accept()` returns a **new** socket; why `recv()` is used in a **loop**; the difference between `send` and `sendall`; what `SO_REUSEADDR` is for; why bytes and `encode/decode`.

Check yourself

1. What do the two pairs of parentheses in `bind((HOST, PORT))` mean? (*one argument: the tuple (host, port)*)
2. What does `accept()` return? (*a new socket for the client and the client's address*)
3. What would happen if the client were started before the server? (*`ConnectionRefusedError`, caught in lines 13–14*)
4. Why does the client loop on `recv`? (*TCP is a byte stream: one `recv` may deliver only a part; an empty result means the server has closed*)
5. Why `b"File not found"` and not `"File not found"`? (*sockets send bytes*)
6. Which statements make the server stop cleanly on Ctrl+C? (*`except KeyboardInterrupt` and `finally: server_socket.close()`*)

EXPERIMENT 9

Experiment 9: Client-Server Calculator Using UDP Sockets

What the programs do and why

The **client** asks the user for two numbers and an operator, checks them, and sends them in **one datagram** such as `12 + 5` to the server. The **server** calculates and sends the result back in another datagram (`17.0`) or an error text. The client shows the result, or an *appropriate message for invalid input* (the task of the experiment). Both programs use **UDP** (`SOCK_DGRAM`): there is no connection, no `listen()`, no `accept()` and no `connect()`; every message is an independent datagram that carries the address of its sender.

The conversation (the application protocol)

1. Client: read `num1`, `operator`, `num2`; reject invalid input *without sending anything*.
2. Client sends the text `num1 operator num2` (one datagram) and waits at most 5 seconds.
3. Server: receives the datagram and learns the client's address; splits it into three parts, converts the numbers, calculates; builds `result` or an `Error: ...` text.
4. Server sends `str(result)` as one datagram to that address; the client prints `12 + 5 = 17.0`.



Figure A4. In UDP there is one socket on each side and no connection. The server replies to the address that `recvfrom()` returned.

Line by line: udp_server.py

```
1 # udp_server.py - receives "number operator number" and replies with the result
2 import socket
```

A comment and `import socket`: the only module the server needs.

```
4 SERVER_IP = "127.0.0.1"
5 SERVER_PORT = 20001
6 BUFFER_SIZE = 1024
```

Constants: the server's address `127.0.0.1` (this computer), the port **20001** and `BUFFER_SIZE = 1024`, the largest datagram the server will read in one `recvfrom`.

```
8 # create a UDP socket and bind it to (SERVER_IP, SERVER_PORT)
9 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10 server_socket.bind((SERVER_IP, SERVER_PORT))
```

Creates a **datagram socket**: `SOCK_DGRAM` means UDP (`AF_INET` = IPv4). Then `bind((SERVER_IP, SERVER_PORT))` attaches it to the address, so that clients know where to send. Notice what is *missing* compared with the TCP server: there is **no `listen()` and no `accept()`**, because UDP has no connections.

```
11 print("UDP server up and listening on port", SERVER_PORT)
```

Prints a message that the server is ready.

```
13 try:
14     while True:
```

`try:` (its `except` and `finally` are on lines 48–51) and `while True:`, an infinite loop: the server answers one request after the other.

```
15         # wait for a datagram; recvfrom() also tells us who sent it
16         data, client_address = server_socket.recvfrom(BUFFER_SIZE)
```

`recvfrom(BUFFER_SIZE)` **waits** until a datagram arrives and returns a pair `(data, address)`: the **bytes** of that one datagram and the sender's address, e.g. `('127.0.0.1', 42284)`. The pair is unpacked at once into `data` and `client_address`. One `recvfrom` always returns exactly one datagram (message boundaries are kept in UDP); if a datagram were longer than the buffer, the excess would be discarded (Windows reports an error instead).

```
17         request = data.decode(errors="replace")
18         print("Request from", client_address, ":", request)
```

`data.decode(errors="replace")` converts the bytes to a string; `errors="replace"` replaces bytes that are not valid text instead of raising an error, so garbage from the network can never crash the server. Line 18 prints who sent what.

```
20         # work out the result (or an error message)
21         try:
22             num1, operator, num2 = request.split()
```

`try:` protects the calculation. `request.split()` cuts the text at the blanks into a list, e.g. `['12', '+', '5']`. `num1, operator, num2 = ...` **unpacks exactly three items**; if the list has any other length (`hello`, `3 +`) Python raises `ValueError`.

```
23             a = float(num1)
24             b = float(num2)
```

`float(num1)` and `float(num2)` convert the text to real numbers; text such as `abc` raises `ValueError` as well. Both kinds of error are caught on lines 43–44.

```
25             if operator == "+":
26                 result = a + b
27             elif operator == "-":
28                 result = a - b
29             elif operator == "*":
30                 result = a * b
```

The operator decides: `+`, `-` and `*` give `result = a + b` and so on. `elif` tests the next case only if the previous ones failed.

```
31             elif operator == "/":
32                 if b == 0:
33                     result = "Error: division by zero"
34                 else:
35                     result = a / b
```

For `/`: **division by zero is checked first**. If `b == 0` the result is the *text* `Error: division by zero`; otherwise `a / b`.

```
36             elif operator == "%":
37                 if b == 0:
38                     result = "Error: modulo by zero"
39                 else:
40                     result = a % b
```

The same for `%` (remainder). Without the check, `7 % 0` raises `ZeroDivisionError` and the server would stop.

```
41              else:
42                  result = "Error: invalid operator"
```

`else:` any other operator: the result is the text `Error: invalid operator`.

```
43          except ValueError:
44              result = "Error: invalid request"
```

`except ValueError:` is executed if line 22, 23 or 24 raised a `ValueError`: the request was malformed, so the result is `Error: invalid request`.

```
46          print("Result to client:", result)
47          server_socket.sendto(str(result).encode(), client_address)
```

Prints the result on the server screen. Then `str(result).encode()` converts the number or the message to text and then to **bytes**, and `sendto(bytes, client_address)` sends **one datagram** to the address that `recvfrom` returned. A UDP socket is not connected, so every `sendto` must name the destination.

```
48 except KeyboardInterrupt:
49     print("\nServer stopped")
50 finally:
51     server_socket.close()
```

Ctrl+C is caught (`Server stopped`), and the `finally` block always closes the socket, as in the TCP server.

Line by line: udp_client.py

```
1 # udp_client.py - reads two numbers and an operator, asks the server to calculate
2 import socket
3 import sys
4 import math
```

Comment and three imports: `socket` (network), `sys` (`sys.exit`) and `math` (`math.isfinite`).

```
6 SERVER_ADDRESS = ("127.0.0.1", 20001) # (IP address, port) of the server
7 BUFFER_SIZE = 1024
```

The **address of the server** (it must be the same as in the server program) and the buffer size for the reply.

```
9  def is_number(text):
10     # True only for finite numbers such as 12, -3.5 or 1e3
11     try:
12         return math.isfinite(float(text))
13     except ValueError:
14         return False
```

`def is_number(text):` is a small function that answers *is this text a usable number?* `float(text)` converts the text and raises `ValueError` if it is not a number; the `try/except` turns that into `False`. `math.isfinite(...)` is true for normal numbers but false for `inf` and `nan`, which `float` would otherwise accept (`float("nan")` works!). The function returns `True` or `False`.

```
16 # read and check the input before anything is sent
17 num1 = input("Enter first number : ").strip()
18 operator = input("Enter operator (+ - * / %) : ").strip()
19 num2 = input("Enter second number : ").strip()
```

Reads the three items. `input(...)` returns **strings**; `.strip()` removes blanks at both ends. The operator is read with `input`, *not* as a command-line argument, because the shell would expand `*` into a list of file names (shown below).

```
21 if not is_number(num1) or not is_number(num2):
22     sys.exit("Invalid input: both operands must be numbers.")
```

`if not is_number(num1) or not is_number(num2):` at least one operand is not a number: `sys.exit("...")` prints the message and ends the program. **Nothing is sent to the server** in this case.

```
23 if operator not in ("+", "-", "*", "/", "%"):
24     sys.exit("Invalid operator: use one of + - * / %")
```

`operator not in ("+", "-", "*", "/", "%")` tests whether the operator is one of the five allowed ones (membership in a tuple). If not, print the message and stop.

```
26 # create a UDP socket; wait at most 5 seconds for the reply
27 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
28 client_socket.settimeout(5)
```

Creates the **UDP socket** (`SOCK_DGRAM`). There is no `connect()` and no `bind()`: the operating system chooses a source port at the first `sendto`. `settimeout(5)` means that a blocking call such as `recvfrom` will give up after **5 seconds** and raise `socket.timeout` instead of waiting for ever (UDP is unreliable: the request or the reply may be lost, or the server may not be running).

```
30 request = num1 + " " + operator + " " + num2
31 client_socket.sendto(request.encode(), SERVER_ADDRESS)
```

Builds the request as text, `num1 + " " + operator + " " + num2`, e.g. `12 + 5` (the blanks are what the server's `split()` needs), and sends it as **one datagram**: `sendto(bytes, address)`.

```
33 try:
34     reply, server_address = client_socket.recvfrom(BUFFER_SIZE)
35     print(request, "=", reply.decode())
```

`recvfrom(BUFFER_SIZE)` waits (at most 5 s) for the reply and returns `(data, address)`; here `reply, server_address = ...`. `reply.decode()` turns the bytes into text and `print(request, "=", ...)` shows e.g. `12 + 5 = 17.0`. If the server sent an error text, that text is shown after the `=`.

```
36 except (socket.timeout, ConnectionResetError):
37     print("No reply from the server. Is udp_server.py running?")
```

`except (socket.timeout, ConnectionResetError):` no reply came within 5 seconds (or, on Windows, the system reported that nothing listens at the port): print a helpful message.

```
38 finally:
39     client_socket.close()
```

`finally:` always closes the socket, whether a reply arrived or not.

Dry run

Variable	`12 + 5`	`5 / 0`	**raw datagram** `hello`
`data` (server, line 16)	`b'12 + 5'`	`b'5 / 0'`	`b'hello'`
`request.split()` (line 22)	`['12','+','5']`	`['5','/','0']`	`['hello']`: unpack fails
`a`, `b` (lines 23–24)	`12.0`, `5.0`	`5.0`, `0.0`	— (`ValueError`)
`result`	`17.0`	`'Error: division by zero'`	`'Error: invalid request'`
sent (line 47)	`b'17.0'`	`b'Error: division by zero'`	`b'Error: invalid request'`
client prints	`12 + 5 = 17.0`	`5 / 0 = Error: division by zero`	—

Input that is wrong at the client never reaches the server: for the operands `abc` and `5` the client prints `Invalid input: both operands must be numbers.` and stops; for the operator `^` it prints `Invalid operator: use one of + - * / %`.

Why the operator is read with input() (real output): the shell replaces * by the file names of the folder

```
$ echo 6 * 3
6 udp_client.py udp_server.py 3
```

Why this version differs from the sample program of the lab manual

The college sample program reads one expression, for example `12+5`, and cuts it with a regular expression. It works for simple correct input only; with the other input a user can type, the **server stops** and the **client waits for ever**. These runs are real (Python 3.12):

Lab-manual UDP server: three inputs that make it crash

```
## UDP, lab-manual server, input  -5+3   (a negative number)
server side:
    File "udp_server_c.py", line 17, in <module>
      num1, num2 = re.split('\+|-|\*|/|%', expression.replace(' ', ''))
      ^^^^^^^^^^
  ValueError: too many values to unpack (expected 2)
client side: waits for ever (killed after 3 s)

## UDP, lab-manual server, input  abc   (text instead of numbers)
server side:
    File "udp_server_c.py", line 17, in <module>
      num1, num2 = re.split('\+|-|\*|/|%', expression.replace(' ', ''))
      ^^^^^^^^^^
  ValueError: not enough values to unpack (expected 2, got 1)
client side: waits for ever (killed after 3 s)

## UDP, lab-manual server, input  7%0   (modulo by zero)
server side:
    File "udp_server_c.py", line 33, in <module>
      result = num1 % num2
               ~~~~~^~~~~
  ZeroDivisionError: float modulo
client side: waits for ever (killed after 3 s)
```

Problem	Lab-manual version	This version
no invalid-input messages at the client	the client sends whatever is typed	the client checks operands and operator (lines 9–24)
negative number `-5+3` or text `abc` crashes the server	`num1, num2 = re.split(…)` expects exactly two parts	`split()` into three parts and `except ValueError` (lines 22–44)
`7 % 0` crashes the server	no check for `%`	lines 36–40
the client waits for ever	no time-out	`settimeout(5)` (client line 28)
`SyntaxWarning: invalid escape sequence '\+'`	ordinary string `'\+|-|\*|/|%'`	no regular expression is needed
`replace('', '')` does nothing	(the intended blank is missing)	`split()` ignores blanks

Careful. The *Algorithm* printed with the UDP experiment in the college manual lists `connect()`, `listen()` and `accept()` and calls the figure "TCP socket"; its *Theory* says "Refer to Experiment No.

4". All of this is wrong for UDP. In an exam write the UDP calls as: **server** `socket`, `bind`, `recvfrom`, `sendto`, `close`; **client** `socket`, `sendto`, `recvfrom`, `close`.

Change it: what the examiner may ask

Task	Change	Where
other port	`SERVER_PORT` / the port in `SERVER_ADDRESS`	server 5, client 6
a new operator, e.g. power	server: `elif operator == "^": result = a ** b`; client: add `"^"` to the tuple	server 41, client 23
integer results	show `int(result)` when `result == int(result)`	server 46
round the result	`result = round(result, 2)` before sending	after line 42
longer or shorter time-out	`settimeout(2)`	client 28
several calculations in one run	put lines 17–37 of the client into a `while True:` loop	client 17–37
server on another computer	`SERVER_IP = "0.0.0.0"`; client uses the server's address	server 4, client 6

In the exam. Be ready to answer: *Why no `listen()`/`accept()` in UDP?* (no connection). *How does the server know where to reply?* (`recvfrom` returns the sender's address). *Why a time-out in the client?* (datagrams can be lost). *Max UDP payload?* (65 507 bytes over IPv4). *Where is invalid input detected?* (operands and operator at the client; division by zero and malformed requests at the server).

Check yourself

1. Which two methods replace `send` and `recv` in UDP? (*`sendto` and `recvfrom`*)
2. What does `recvfrom` return? (*the data as bytes and the sender's address as a tuple*)
3. What happens on line 22 of the server if the client sends `3 +`? (*unpacking fails with `ValueError`, caught: `Error: invalid request`*)
4. Why does the result show `17.0` and not `17`? (*the operands are converted with `float`*)
5. What does `settimeout(5)` change? (*a blocking call raises `socket.timeout` after 5 seconds*)
6. Why must the client build the request with blanks between the three items? (*the server splits at blanks*)

EXPERIMENT 10

Experiment 10: Distance Vector Routing Algorithm

What the program does and why

Every router must know, for each destination, **how far** it is and **which neighbour** to send the packet to (the **next hop**). In **distance vector routing** a router starts with the costs of its own links only; it then learns from its neighbours' tables and keeps a route through a neighbour whenever *cost to the neighbour + neighbour's distance to the destination* is smaller than what it has. This is the **Bellman–Ford** rule $D_x(y) = \min_v\{c(x,v) + D_v(y)\}$. The program reads the link costs of all routers as a matrix, repeats this rule for every router, destination and intermediate router, and finally prints each router's table: destination, next hop (*via*) and distance.

The method

1. Read n and the $n \times n$ cost matrix `dmat`; 999 means "no link", the diagonal is 0.
2. Start every table with the direct costs: `dist[j] = dmat[i][j]`, next hop `from[j] = j`.
3. Repeat $n - 1$ times: for all `i`, `j`, `k`, if going from `i` to `k` and then from `k` to `j` is cheaper, store the new distance and set the next hop to `k`.
4. Print the table of every router; router 1 is shown as A, 2 as B, and so on.

Line by line: exp10.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
```

The two headers: `stdio.h` for `printf`/`scanf`; `stdlib.h` is not needed by this program but is harmless.

```
3  struct node
4  {
5      int dist[20];
6      int from[20];
7  } rt[10];
```

Declares a **structure** `node` = one routing table: `dist[j]` is the best known distance to router `j`, `from[j]` the next hop used to reach it. `rt[10]` creates an array of ten tables, one per router, as a **global** variable (so every entry starts at 0). Index 0 is not used because the routers are numbered from 1, so at most 9 routers fit.

```
8  int main()
9  {
10     int dmat[20][20];
11     int n,i,j,k,c=1;
```

`dmat[20][20]` holds the direct link costs as typed. `n` is the number of routers, `i`, `j`, `k` are loop counters and `c = 1` counts the rounds of the update loop.

```
12     printf("Enter the number of nodes: \n");
13     scanf("%d", &n);
14     printf("Enter the cost matrix: ");
```

Asks for and reads the number of routers, then prints the prompt for the matrix. The `\n` after "nodes:" moves the cursor to the next line, which is why the number is typed below the prompt.

```
15     for(int i=1; i<=n; i++)
16     {
17         for(int j=1; j<=n; j++)
18         {
```

Two nested loops visit every cell of the matrix, row `i` = router, column `j` = destination. Note that `int i` and `int j` declare **new** loop variables that hide the ones of line 11 inside these loops (allowed in C99 and later).

```
19             scanf("%d", &dmat[i][j]);
20             dmat[i][i]=0;
21             rt[i].dist[j]=dmat[i][j];
22             rt[i].from[j]=j;
```

`scanf` reads one cost. `dmat[i][i]=0` forces the cost from a router to itself to 0 whatever was typed (it runs for every `j`, so it also corrects the diagonal value just after it is read). The table of router `i` is initialised with the direct cost and with next hop = the destination itself (*the router sends directly*).

```
23         }
24     }
25     do
26     {
```

Closes the input loops and opens a **do–while** loop: its body runs at least once and the test is at the end (line 37).

```
27          for(i=1; i<=n; i++)
28              for(j=1; j<=n; j++)
29                  for(k=1; k<=n; k++)
```

Three nested loops: `i` = the router whose table is updated, `j` = the destination, `k` = the neighbour (intermediate router) tried. That is n^3 tests per round.

```
30                      if(rt[i].dist[j]>dmat[i][k] + rt[k].dist[j])
```

The **Bellman–Ford test**: is the cost of the link `i`→`k` (`dmat[i][k]`) plus the distance that `k` knows to `j` (`rt[k].dist[j]`) smaller than the current distance of `i` to `j`? A missing link (999) is never chosen because the sum stays large.

```
31                      {
32                          rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
33                          rt[i].from[j]=k;
34                      }
```

If yes, the table of `i` is updated: the new distance and `from[j] = k`, i.e. "to reach `j`, go through `k`". Line 32 adds `rt[i].dist[k]` instead of `dmat[i][k]`; because the best distance to `k` is never larger than the direct cost, the stored value is still a real path cost and the result is the same for the networks of the manual.

```
35          c++;
36      }
37      while(c<n);
```

`c++` counts the rounds; the loop repeats while `c < n`, so it runs $n-1$ times (2 times for 3 routers, 3 times for 4). $n-1$ rounds are enough because a shortest path has at most $n-1$ links.

```
38      for(i=1; i<=n; i++)
39      {
40          printf("\nDistance Table for router %c is in \n",i+64);
```

For every router a heading is printed. `%c` with `i+64` prints a **character**: ASCII 65 is A, 66 is B, so router 1 appears as "router A".

```
41          for(j=1;j<=n;j++)
42              printf("\tNode %d Via %d.  Distance: %d.\n",j,rt[i].from[j], rt[i].dist[j]);
```

One line per destination: `\t` indents the line, then the destination number, the next hop `from[j]` and the distance `dist[j]`.

```
43      }
44      return 0;
45  }
```

Closes the print loop; `return 0` tells the operating system that the program ended normally (shown as "Process returned 0" in Code::Blocks).

Dry run: the four-router network of the manual

Costs: A–B 2, A–D 1, B–C 3, B–D 7, C–D 11, no link A–C (999). The first round already finds every improvement; rounds 2 and 3 change nothing (real trace of the program):

Every update made by lines 30–33 (real trace, n = 4)

```
round 1: i=1 j=3 k=2   999 > 2+3   -> A to C = 5 via B
round 1: i=2 j=4 k=1     7 > 2+1   -> B to D = 3 via A
round 1: i=3 j=1 k=2   999 > 3+2   -> C to A = 5 via B
round 1: i=3 j=4 k=2    11 > 3+3   -> C to D = 6 via B
round 1: i=4 j=2 k=1     7 > 1+2   -> D to B = 3 via A
round 1: i=4 j=3 k=1    11 > 1+5   -> D to C = 6 via A
round 2: no change
round 3: no change
```

D to C uses the value 5 that A found for C a moment earlier in the same round (A's row is updated before D's), which is why one round is enough here. In the laboratory run with 3 routers (rows `1 0 1`, `2 5 3`, `3 1 0`) no test succeeds, so every router keeps its direct costs.

Change it: what the examiner may ask

Task	Change	Where
show the next hop as a letter	`"Via %c."` with `rt[i].from[j]+64`	line 42
print the number of rounds	print `c` after the loop	after line 37
stop when nothing changes	a flag set to 1 on every update; `while(flag)`	lines 25–37
use the textbook update	`rt[i].dist[j]=dmat[i][k]+rt[k].dist[j];`	line 32
more than 9 routers	enlarge `rt[10]` and `dist/from[20]`	lines 5–7
name "infinity"	`#define INF 999` and type 999 for missing links	top

> **In the exam.** Compare with **link state** routing (every router knows the whole map and runs Dijkstra) and name the weakness of distance vector: **count-to-infinity** after a link failure, reduced by *split horizon* and *poison reverse*. RIP is the classic distance vector protocol.

Check yourself

1. What do `dist[j]` and `from[j]` store? (*best distance to j and the next hop for j*)
2. Why is `dmat[i][i]` set to 0? (*a router reaches itself at no cost*)
3. How many times does the do–while loop run for 4 routers? (*3, that is $n-1$*)
4. What does `printf("%c", 3+64)` print? (*C*)
5. Why is 999 used for a missing link? (*it acts as infinity and is replaced as soon as a path exists*)
6. In the 4-router network, through which router does D reach C, and at what cost? (*via A, 6*)

PART C

Exam Toolkit: Formulas, Commands, Cross-Checks

One-page cheat sheet

Topic	Formula or rule
CBR rate	`packetSize_` $\times$ 8 / `interval_`: 500 $\times$ 8 / 0.005 = 800 kb/s
transmission time	size $\times$ 8 / bandwidth: 500 B on 1 Mb/s = 4 ms
time on one link	transmission time + propagation delay: 4 + 10 = 14 ms
queue limit	DropTail drops the *arriving* packet; holds at most limit $-$ 1 waiting packets
drops (overload)	$\approx N - (C/L)\,T -$ limit (N packets sent, C bit/s, L bits per packet, T seconds)
throughput in the AWK scripts	(bytes received / (last receive $-$ first enqueue)) $\times$ 8/1000 = kb/s
trace fields	`$1` event (+ $-$ r d h), `$2` time, `$3`/`$4` from/to node, `$5` type, `$6` size, `$8` flow id, `$11` sequence, `$12` packet id
slow start / avoidance	window doubles per round trip / grows by about one packet per round trip
bit stuffing	insert 0 after five 1s; the receiver removes the 0 after five 1s; flag 01111110; worst case 20 %
CRC	append $m-1$ zeros, XOR the generator under every leading 1, the last $m-1$ bits are the CRC; receiver: all zero $\Rightarrow$ no error; CRC-CCITT: 0x11021
bubble sort	n(n$-$1)/2 comparisons; after pass i the i largest items are in place
TCP server / client calls	socket, bind, listen, accept, recv, sendall, close / socket, connect, sendall, recv, close
UDP server / client calls	socket, bind, recvfrom, sendto, close / socket, sendto, recvfrom, close
max UDP payload (IPv4)	65 535 $-$ 20 (IP) $-$ 8 (UDP) = 65 507 bytes

NS-2 commands at a glance

Command	What it does
`set ns [new Simulator]`	creates the simulator object
`$ns namtrace-all $nf` / `$ns trace-all $tf`	write the NAM file / the text trace file
`set n0 [$ns node]`	creates a node
`$ns duplex-link a b 1Mb 10ms DropTail`	two-way link: bandwidth, delay, queue type
`$ns queue-limit a b 50`	buffer of the link from a to b, in packets
`$ns duplex-link-op a b orient right`	layout for NAM only
`new Agent/UDP`, `Agent/Null`, `Agent/TCP`, `Agent/TCPSink`	transport endpoints (senders and sinks)
`$ns attach-agent $n $agent`	puts an agent on a node
`$ns connect $src $dst`	sets the destination of the sender
`new Application/Traffic/CBR`, `Application/FTP`	traffic sources: constant rate / bulk transfer
`$obj attach-agent $agent`	connects an application to its agent
`$ns at 0.5 "$cbr0 start"`	schedules a command at a simulated time
`$ns make-lan "nodes" 2Mb 40ms LL Queue/DropTail Mac/802_3`	builds an Ethernet LAN
`$ns color 1 red` / `$tcp set class_ 1`	NAM colour of a flow / flow id of the agent
`$ns lossmodel $em a b`	places an error model on the link a to b
`$tcp attach $file` and `$tcp trace cwnd_`	log every change of the congestion window
`$ns run`	starts the simulation

Typing checklist for the NS-2 experiments

- Braces: every `{` has its `}`; the `{` of `proc finish {} {` is on the same line.
- Inside `finish`: `global` lists every variable that the procedure uses (also `tfile1 tfile2` in Experiment 4).
- The whole `make-lan` command is on **one line**; class names are spelt exactly (`Mac/802_3`, `Queue/DropTail`).
- Option names are exact: `packetSize_`, `interval_`, `class_`, `cwnd_` (a wrong name is silently accepted for `set`).
- `$ns at time "command"` in **double quotes**; times in seconds; `finish` last.
- No stray characters on lines of their own; Tcl would try to execute them.
- Run `ns file.tcl`; if nothing appears, check that you did not leave the `%` prompt of an interactive `ns` (Ctrl-D).
- For AWK: the `{` after `BEGIN` on the same line; double quotes; every `(` closed.

Cross-checks that were run for this guide

Everything in the guide was checked in one of three ways. The Python programs and the AWK scripts were **executed** and their real output is shown. The NS-2 facts were compared with the **NS-2 manual** and the Python facts with the **Python documentation** (see the sources). Finally, three results of the manual were **recomputed independently**:

- *Experiment 1:* a model of a DropTail queue (below) reproduces the manual's eight drop counts to within one packet (Figure A2).
- *Experiment 6:* the division loop of `exp6.py`, applied to the ASCII text `123456789` with the 17-bit generator, gives 0x31C3, the published check value of CRC-16/XMODEM, and the 20-bit example of the manual gets the same 16 check bits as in the manual.
- *Experiments 5, 7, 8 and 9:* all programs were run with the inputs of the manual; the outputs agree. For the sockets additional runs show the error cases.

The queue model used for the check of Experiment 1 (Python)

```
def drops(C_bps, limit, pkt=500, interval=0.005, t0=0.5, n=801):
    s = pkt * 8 / C_bps                  # service (transmission) time of one packet
    in_system = []                       # departure times of packets still in the system
    last = 0.0; dropped = 0
    for k in range(n):                   # packet k arrives at t0 + k * interval
        t = t0 + k * interval
        in_system = [d for d in in_system if d > t]
        if len(in_system) >= limit:      # 1 in service + (limit - 1) waiting: full
            dropped += 1
            continue
        start = max(t, last); last = start + s; in_system.append(last)
    return dropped

drops(0.5e6, 50)   ->  252     drops(0.25e6, 50) -> 502     drops(1e6, 50) -> 0
```

Where the lab manual is unclear or wrong

These points were found while checking the programs; none of them changes the results of your record, but an examiner may ask.

- **Experiment 2, line 62:** `Agent/TCP set packetSize_ 1000` comes *after* the TCP agent was created, so it changes only agents created later; harmless because 1000 is the default.
- **Experiment 2, line 30:** the label "Receiver" is put on the router `n2`; the real receiver is `n3`.
- **Experiment 3, line 17:** the comment says six nodes, but seven are created.
- **Experiment 4:** the aim mentions *collisions*, but the script does not count them.
- **College manual, UDP experiment:** the *Algorithm* lists `connect()`, `listen()` and `accept()`, the figure is captioned "TCP", and the *Theory* refers to Experiment 4; the sample programs contain the bugs shown in Experiments 8 and 9.
- **College manual, TCP client:** compares a string with a bytes value, so *File not found* is never reported.

Exam-day checklist

1. Write the **call order** or the **steps** first (they earn marks), then the code.
2. Start the server before the client; use two terminals in the same folder.

3. Test the error cases: missing file, division by zero, wrong input.
4. Save each result before the next run (`cp`), and name the files as in Appendix C of the manual.
5. If asked to change a parameter, use the *Change it* tables of the chapters.

REFERENCES

Sources Used to Check the Facts

The facts of this guide were checked against these pages (opened while preparing the guide). The output of every program shown was produced by running it.

- **Python sockets.** Socket Programming HOWTO: `https://docs.python.org/3/howto/sockets.html`; socket objects (`send`, `sendall`, `recv`, `recvfrom`, `sendto`): `https://docs.python.org/2.5/lib/socket-objects.html`.
- **Address already in use / SO_REUSEADDR.** `https://bobbyhadz.com/blog/socket-error-errno-98-address-already-in-use-in-python`; the difference on Windows: `https://bugs.python.org/msg150511`.
- **UDP.** Header and payload size: `https://networklessons.com/ip-routing/user-datagram-protocol-udp-packet-header`; `https://www.oreilly.com/library/view/windows-server-2008/9780735624474/ch09s03.html`.
- **NS-2 trace file format** (twelve fields, events + - r d): `https://www.isi.edu/nsnam/ns/doc/node289.html`.
- **NS-2 CBR source** (`rate_`, `interval_`, `packetSize_`, `random_`): `https://isi.edu/websites/nsnam/ns/doc/node510.html`; applications: `https://isi.edu/nsnam/ns/doc/node516.html`.
- **NS-2 DropTail and queue limit** ("qlim_ set to n means a queue size of n−1"): `https://www.isi.edu/nsnam/ns/doc/node68.html`; the `ns` command reference: `https://manpages.ubuntu.com/manpages/kinetic/man1/ns.1.html`.
- **Header added by the NS-2 TCP agent** (40 bytes, `tcpip_base_hdr_size_`; UDP adds none): `https://www.isi.edu/nsnam/archive/ns-users/webarch/2000/msg01441.html`.
- **NS-2 TCP parameters** (`window_`, `windowInit_`, `packetSize_`): `https://www.isi.edu/nsnam/ns/doc/node396.html`; agents: `https://www.isi.edu/websites/nsnam/ns/doc/node119.html`.
- **NS-2 error models** (`unit`, `rate_`, `ranvar`, `drop-target`, `lossmodel`): `https://www.isi.edu/nsnam/ns/doc/node138.html`; `https://www.isi.edu/websites/nsnam/ns/doc/node141.html`.
- **CRC-CCITT variants** (polynomial 0x1021; XMODEM start value 0, check value 0x31C3): `https://blog.mbedded.ninja/programming/general/crcs-cyclic-redundancy-checks`; `https://docs.rs/crate/crc16`.
- **Bit stuffing and the HDLC flag 01111110:** `https://farsite.com/product-support/network-interface-connector-standards/hdlc-framing-standard/`; `https://www.techtarget.com/searchnetworking/definition/bit-stuffing`.